1. A student has received marks in four subjects and wants to predict their total and average marks based on potential improvements in **Subject 4**.

Given Data:

| Subject | Marks |
|---|---|
| Subject 1 | 80 |
| Subject 2 | 75 |
| Subject 3 | 85 |
| Subject 4 | 70 |

Tasks:

1. **Calculate** the total marks and average marks using Excel formulas.
2. Use **What-If Analysis (Data Table)** to examine how the total marks change if the student scores different marks in **Subject 4** (e.g., 70, 75, 80, 85, 90, 95).
3. **Determine** the minimum marks the student must score in **Subject 4** to achieve an **average of 80** using **Goal Seek**.

18. A student has scored marks in four subjects, and they want to predict their **total and average marks** based on possible improvements in **Subject 4**.

Given Data:

Subject Marks

Subject 1 78

Subject 2 82

Subject 3 74

Subject 4 65

1. Calculate the **total marks** and **average marks** using Excel formulas.
2. Use **What-If Analysis (Data Table)** to check how the **total marks** will change if the student scores **different marks in Subject 4 (e.g., 65, 70, 75, 80, 85, 90)**.

Find out the minimum marks the student must score in **Subject 4** to achieve an **average of 80** using **Goal Seek**.

| A | B |
|---|---|
| **Subject** | **Marks** |
| sub 1 | 80 |
| sub 2 | 75 |
| sub 3 | 85 |
| sub 4 | 70 |

## Task 1: Calculate Total and Average Marks

**The Objective:** Establish our baseline metrics. Before we can predict how changes will affect our total and average, we need dynamic formulas that will automatically recalculate when the underlying data changes.

**Steps & Commands:**

1. Click on cell **A6** and type `Total`.
2. Click on cell **B6** and enter the command: `=SUM(B2:B5)`
3. Click on cell **A7** and type `Average`.
4. Click on cell **B7** and enter the command: `=AVERAGE(B2:B5)`

**The Explanation:** We use `=SUM()` and `=AVERAGE()` instead of manually adding the numbers together. In Business Intelligence, we always build dynamic models. If you just typed "310" for the total, our next tasks would fail because the total wouldn't update when we simulate changes to Subject 4.

| | A | B | C | D | E |
|---|---|---|---|---|---|
| 1 | **Subject** | **Marks** | | | |
| 2 | sub 1 | 80 | | | |
| 3 | sub 2 | 75 | | | |
| 4 | sub 3 | 85 | | | |
| 5 | sub 4 | 70 | | | |
| 6 | | | | | |
| 7 | total | 310 | | | |
| 8 | Average | 77.5 | | | |
| 9 | | | | | |

## Task 2: What-If Analysis (Data Table)

**The Objective:** Examine multiple potential scenarios at once. Instead of manually typing 75, then 80, then 85 into cell B5 and writing down the total each time, a Data Table automates this comparison.

**Steps & Commands:**

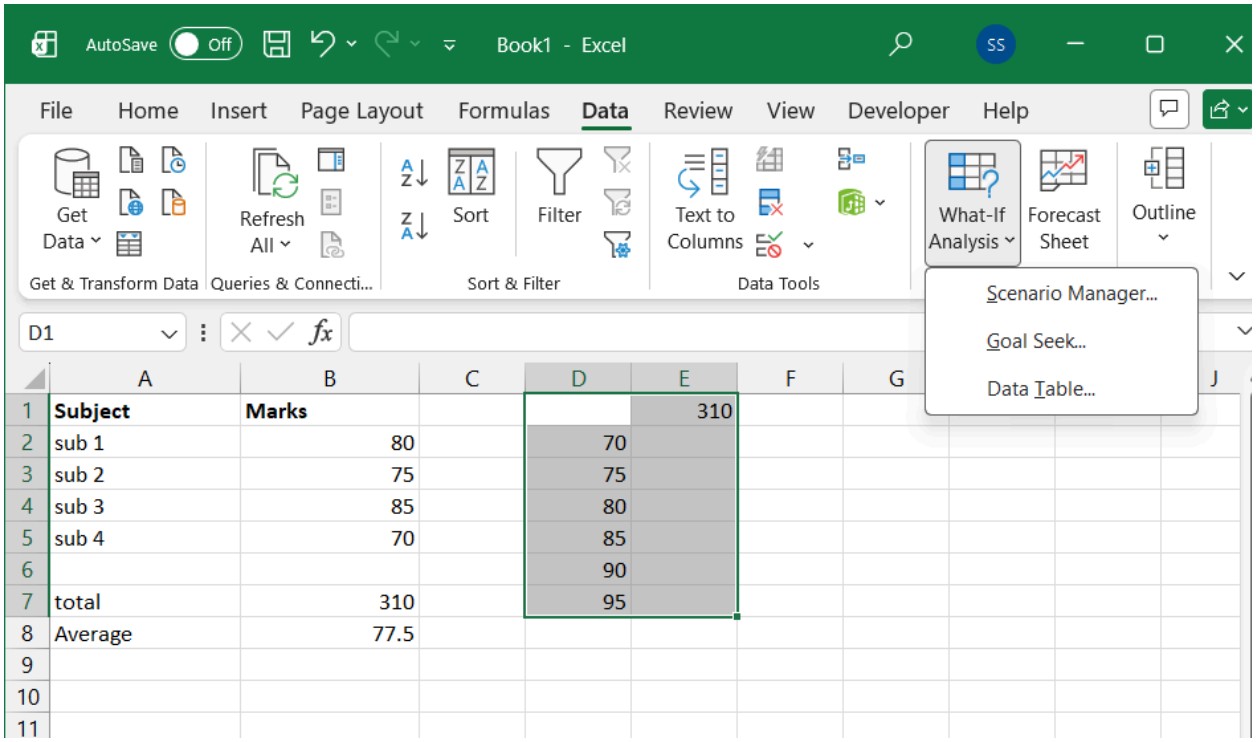
1. Let's create our scenario table. In column **D**, starting from **D2**, enter the potential Subject 4 marks: `70`, `75`, `80`, `85`, `90`, `95` (down to cell D7).
2. In cell **E1** (the column right next to your scenarios, one row above), create a link to your total formula by typing: `=B6`

| | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| 1 | **Subject** | **Marks** | | | =B7 | |
| 2 | sub 1 | 80 | | 70 | | |
| 3 | sub 2 | 75 | | 75 | | |
| 4 | sub 3 | 85 | | 80 | | |
| 5 | sub 4 | 70 | | 85 | | |
| 6 | | | | 90 | | |
| 7 | total | 310 | | 95 | | |
| 8 | Average | 77.5 | | | | |
| 9 | | | | | | |

3. Select the entire range containing your scenarios and the formula header: **D1:E7**.

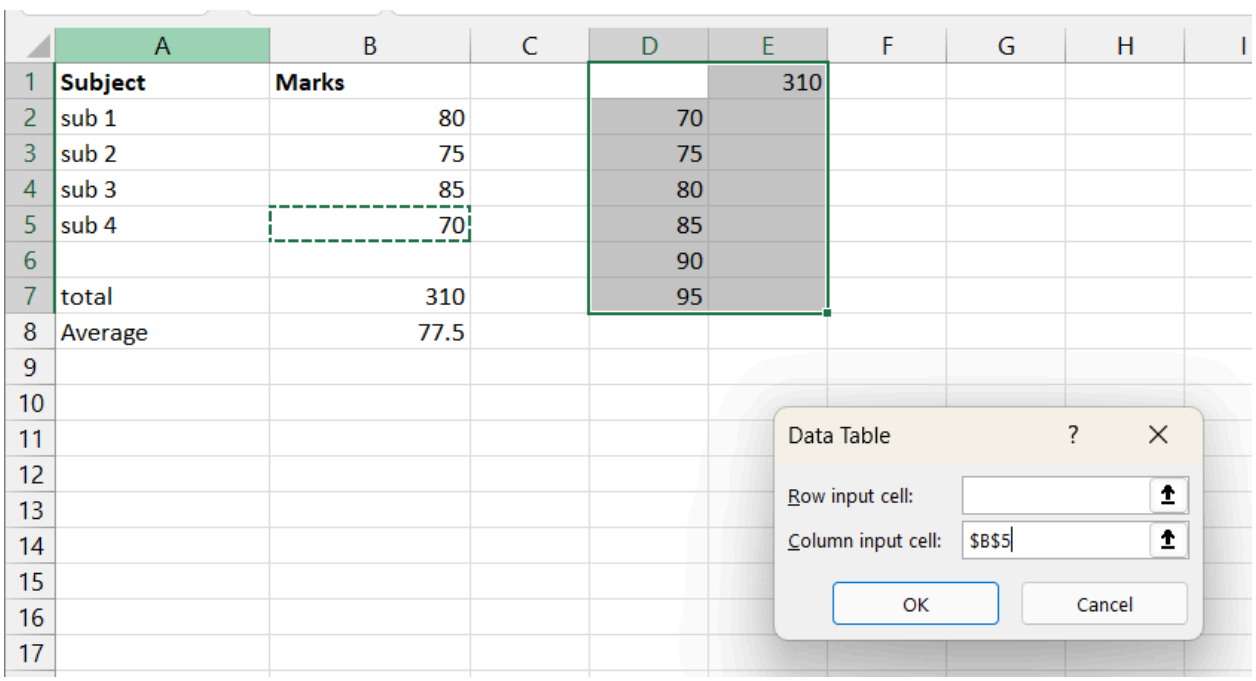
| | C | D | E | F |
|---|---|---|---|---|
| | | | 310 | |
| 80 | | 70 | | |
| 75 | | 75 | | |
| 85 | | 80 | | |
| 70 | | 85 | | |
| | | 90 | | |
| 310 | | 95 | | |
| 77.5 | | | | |

4. On the Excel ribbon, go to the **Data** tab.
5. Click on **What-If Analysis** and select **Data Table…**




6. A small dialog box will appear.

- Leave **Row input cell** blank.
- Click inside **Column input cell** and select cell **B5** (your original Subject 4 mark).




Click **OK**.

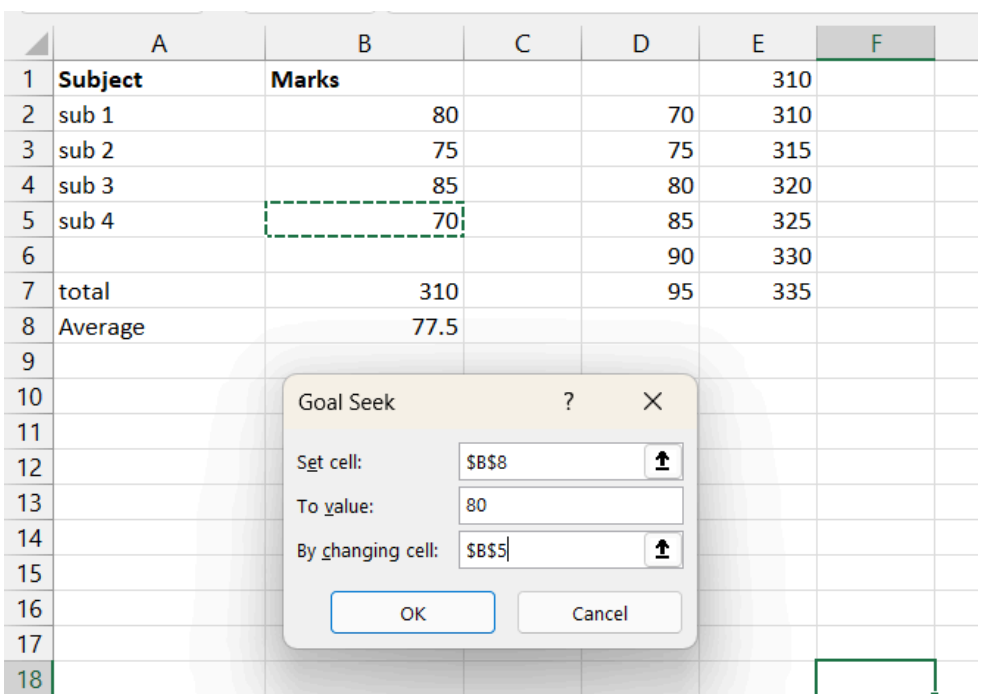
**The Explanation:** Why did we use the *Column* input cell and leave the *Row* input cell blank? Because we listed our "What-If" variations (70, 75, 80...) vertically in a *column*. By selecting B5 as the Column input cell, we are telling Excel: "Take these vertical values one by one, plug them into cell B5 behind the scenes, and output the resulting Total in this table."

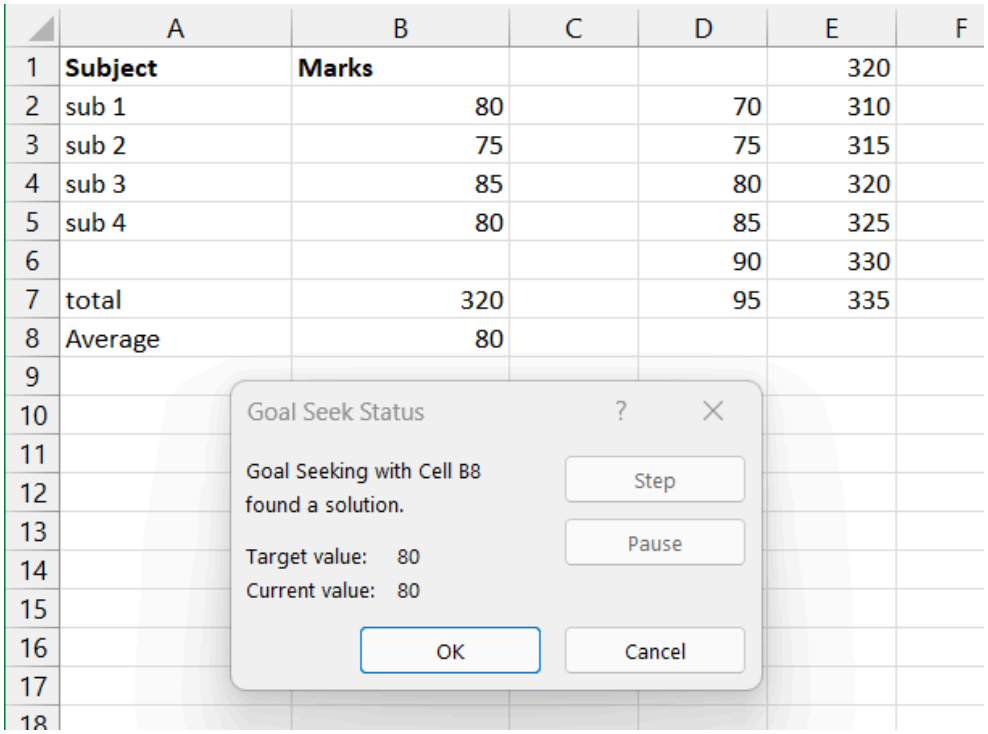
## Task 3: Goal Seek

**The Objective:** Solve a problem backward. Instead of knowing the inputs and calculating the result, we know the *desired result* (an average of 80) and need Excel to find the exact input (Subject 4 marks) required to achieve it.

**Steps & Commands:**

1. Make sure your current Average in cell **B8** is calculated using the `=AVERAGE(B2:B5)` formula.
2. Go to the **Data** tab.
3. Click on **What-If Analysis** and select **Goal Seek...**
4. Fill in the dialog box exactly like this:
    - **Set cell:** `B8` (This is the cell containing the final result you want to change).
    - **To value:** `80` (This is your target average).
    - **By changing cell:** `B5` (This is the specific input cell Excel is allowed to alter).
5. Click **OK**.





| | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| 1 | **Subject** | **Marks** | | | 320 | |
| 2 | sub 1 | 80 | | 70 | 310 | |
| 3 | sub 2 | 75 | | 75 | 315 | |
| 4 | sub 3 | 85 | | 80 | 320 | |
| 5 | sub 4 | 80 | | 85 | 325 | |
| 6 | | | | 90 | 330 | |
| 7 | total | 320 | | 95 | 335 | |
| 8 | Average | 80 | | | | |



**The Explanation:** Excel will rapidly run through calculations until it finds the exact number that needs to be in B5 to make B7 equal 80. Once it finishes, it will overwrite your original 70 in cell B5 with the new required score (which will be 80). Goal Seek is a fundamental prescriptive analytics tool—it tells a business (or a student) exactly what actions they must take to hit a specific KPI target.

| 2. | Show Implementation of Classification algorithm in Python/R. |
|---|---|

For Q2,"Show Implementation of Classification algorithm in Python/R."

We will build a **Decision Tree Classifier** . Decision Trees are fantastic for Business Intelligence because they don't just act like a black box; they effectively create a readable flowchart of rules (like "If revenue > X and churn < Y, then category A") that you can explain to business stakeholders.

We will use the classic Iris dataset from the scikit-learn library. It's a standard dataset perfect for demonstrating classification concepts without getting bogged down in messy data cleaning.

Here is the complete Python implementation, followed by the breakdown of what we are doing and why.

## The Python Implementation

You can write and run this code in any Jupyter Notebook, Google Colab, or your preferred IDE.

```
# 1. Import necessary libraries

import pandas as pd

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, classification_report


# 2. Load the dataset

iris = load_iris()

X = pd.DataFrame(iris.data, columns=iris.feature_names) # Features (measurements)

y = iris.target # Target variable (flower species)


# 3. Split the data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)


# 4. Initialize the Classification Model

clf = DecisionTreeClassifier(random_state=42)


# 5. Train the model

clf.fit(X_train, y_train)


# 6. Make predictions on the test set

y_pred = clf.predict(X_test)
```

```
# 7. Evaluate the model

accuracy = accuracy_score(y_test, y_pred)

print(f"Model Accuracy: {accuracy * 100:.2f}%")

print("\nClassification Report:")

print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

**op**:

```
Model Accuracy: 100.00%

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        19
  versicolor       1.00      1.00      1.00        13
   virginica       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

## Step-by-Step Explanation: The "Why"

**1. Importing Libraries:**

- **Why:** Instead of writing complex mathematical algorithms from scratch, we use `scikit-learn` (sklearn), which is the industry standard for machine learning in Python. It contains pre-built datasets, algorithms, and evaluation tools.

**2. Loading the Dataset:**

- **Why:** A machine learning model needs historical data to learn from. `X` represents our inputs (the independent variables, like petal length). `y` represents our output (the dependent variable, the specific class or category we want to predict).

**3. Splitting the Data (`train_test_split`):**

- **Why:** This is a crucial concept! We never test a model on the exact same data we used to train it. If we did, we wouldn't know if the model actually learned the underlying patterns or just memorized the answers. We hold back 30% (`test_size=0.3`) as unseen data to evaluate its true performance.

**4 & 5. Initializing and Training (fit):**

- **Why:** DecisionTreeClassifier() creates an empty model. The .fit(X_train, y_train) command is where the actual learning happens. The algorithm looks at the training features and targets to figure out the best mathematical rules to separate the different classes.

**6. Making Predictions (predict):**

- **Why:** Now that the model is trained, we give it the 30% of data it has never seen before (X_test) without giving it the answers. We ask it to predict the categories based on the rules it just learned.

**7. Evaluating the Model (accuracy_score):**

- **Why:** In BI, you must validate your models. We compare the model's predictions (y_pred) against the actual hidden answers (y_test). The accuracy score tells us what percentage of the predictions were perfectly correct.

| 4. | Perform the data clustering using clustering algorithm in Python/R. |
|---|---|

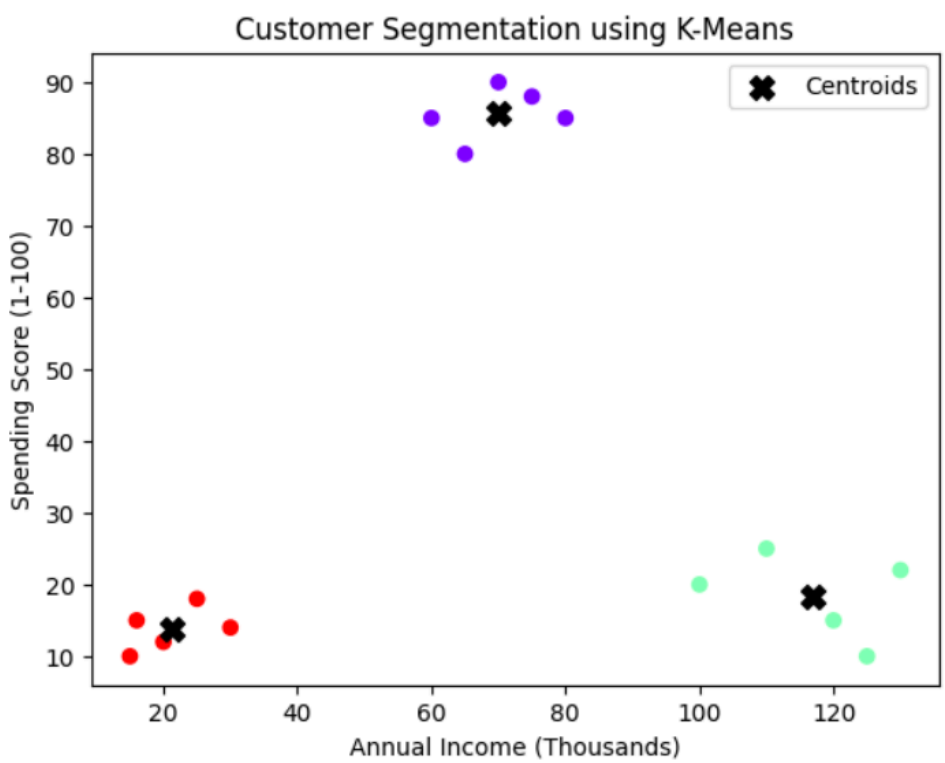
In Question 2, we looked at *Classification*, which is **Supervised Learning**—we already knew the flower species and taught the model to recognize them.

Clustering is **Unsupervised Learning**. This means we have a bunch of data, but we *don't* have any predefined categories or labels. The algorithm's job is to analyze the data and group similar items together organically. In Business Intelligence, this is the exact technology used for **Customer Segmentation** (e.g., grouping customers into "Bargain Hunters," "Big Spenders," and "Loyalists" based on their purchasing habits without manually tagging them).

## The Python Implementation

```
# 1. Import necessary libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# 2. Create sample business data (Customer Income vs. Spending Score)
data = {
    'Annual_Income_k': [15, 16, 20, 25, 30, 60, 65, 70, 75, 80, 100, 110, 120, 125, 130],
    'Spending_Score':  [10, 15, 12, 18, 14, 85, 80, 90, 88, 85, 20, 25, 15, 10, 22]
}
df = pd.DataFrame(data)

# 3. Initialize the K-Means Clustering model
# We are telling the algorithm to find exactly 3 distinct customer segments
```

```
kmeans = KMeans(n_clusters=3, random_state=42)

# 4. Fit the model and assign each customer to a cluster
df['Cluster'] = kmeans.fit_predict(df[['Annual_Income_k', 'Spending_Score']])

# 5. Visualize the resulting clusters
plt.scatter(df['Annual_Income_k'], df['Spending_Score'], c=df['Cluster'], cmap='rainbow')

# Plot the "Centroids" (the mathematical center of each group)
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            color='black', marker='X', s=100, label='Centroids')

plt.title('Customer Segmentation using K-Means')
plt.xlabel('Annual Income (Thousands)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.show()

# Display the data table with their new cluster assignments
print(df)
```



| | Annual_Income_k | Spending_Score | Cluster |
|---|---|---|---|
| 0 | 15 | 10 | 2 |
| 1 | 16 | 15 | 2 |
| 2 | 20 | 12 | 2 |
| 3 | 25 | 18 | 2 |
| 4 | 30 | 14 | 2 |
| 5 | 60 | 85 | 0 |
| 6 | 65 | 80 | 0 |
| 7 | 70 | 90 | 0 |
| 8 | 75 | 88 | 0 |
| 9 | 80 | 85 | 0 |
| 10 | 100 | 20 | 1 |
| 11 | 110 | 25 | 1 |
| 12 | 120 | 15 | 1 |
| 13 | 125 | 10 | 1 |
| 14 | 130 | 22 | 1 |

## Step-by-Step Explanation: The "Why"

**1. Importing Libraries (including `matplotlib`):**

- **Why:** In clustering, visualization is just as important as the math. We use `matplotlib.pyplot` to draw a graph so stakeholders can physically *see* the customer segments.

**2. Creating the DataFrame:**

- **Why:** We are passing two numerical dimensions to the algorithm. K-Means calculates the spatial distance (Euclidean distance) between these numbers to figure out who is similar to whom.

**3. Initializing K-Means (`n_clusters=3`):**

- **Why:** The "K" in K-Means stands for the number of groups you want to create. By setting it to 3, we are forcing the algorithm to divide the entire dataset into 3 specific business profiles.

**4. Fitting and Predicting (`fit_predict`):**

- **Why:** Unlike classification where we split data into training and testing, unsupervised learning looks at the *whole* dataset at once. `fit_predict` tells the algorithm to find the patterns and immediately assign a label (0, 1, or 2) to each row of our data.

**5. Plotting the Centroids:**

- **Why:** A centroid is the literal "center of gravity" for a cluster. The algorithm works by randomly placing these 'X' marks on the graph, measuring which data points are closest to them, and then adjusting the 'X' marks until they sit perfectly in the middle of a dense group.

| | |
|---|---|
| 5. | Perform linear regression on the given data warehouse data using R/Python<br>Perform logistic regression on the given data warehouse data using R/Python |

- **Linear Regression** is used to predict a **continuous numerical value**. Think of a sliding scale. (e.g., "How much revenue will we make if we spend $5,000 on marketing?")
- **Logistic Regression** is used to predict a **categorical value**, usually binary. Think of a light switch—it's either on or off. (e.g., "Will this specific customer cancel their subscription (churn)? Yes or No?")
  Let's do both implementations. I'll use practical Business Intelligence scenarios for each.

## Part A: Linear Regression (Predicting Revenue)

Let's pretend we extracted data from our data warehouse showing how much we spent on Ads and how much Revenue it generated.

```
# 1. Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 2. Create sample data (Ad Spend in thousands vs. Revenue in thousands)
data = {
    'Ad_Spend': [10, 20, 30, 40, 50, 60, 70, 80],
    'Revenue':  [25, 45, 65, 80, 105, 125, 140, 160]
}
df = pd.DataFrame(data)

# 3. Define Features (X) and Target (y)
X = df[['Ad_Spend']]
y = df['Revenue']

# 4. Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 5. Initialize and Train the Model
model = LinearRegression()
model.fit(X_train, y_train)

# 6. Make Predictions
```

```
y_pred = model.predict(X_test)

# 7. Evaluate
print("Linear Regression Results:")
print(f"R-squared Score: {r2_score(y_test, y_pred):.2f}")
```

```
Linear Regression Results:
R-squared Score: 1.00
```

**The Explanation:**

- **Why `LinearRegression()`?** The algorithm mathematically calculates a "line of best fit" through our data points. It finds the relationship (slope) between Ad Spend and Revenue so that if we plug in a brand new Ad Spend number, it can predict the Revenue on that line.
- **Why `r2_score`?** Unlike Classification where we measure "Accuracy" (right or wrong), Linear Regression predictions are rarely exactly to the decimal point. The R-squared score tells us how well our line fits the data points (1.0 is a perfect fit).

---

## Part B: Logistic Regression (Predicting Customer Churn)

Now, let's look at Logistic Regression. This is incredibly useful for predicting whether a customer will leave (churn) based on their engagement metrics.

```
# 1. Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# 2. Create sample data (Months Subscribed, Support Tickets, and Churned status)
# Churn: 1 = Yes (Customer left), 0 = No (Customer stayed)
data = {
    'Months_Subscribed': [12, 1, 24, 2, 36, 3, 48, 1, 6, 2],
    'Support_Tickets':   [0, 5, 1, 4, 0, 6, 2, 7, 1, 5],
    'Churned':           [0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
}
df = pd.DataFrame(data)

# 3. Define Features (X) and Target (y)
X = df[['Months_Subscribed', 'Support_Tickets']]
y = df['Churned']

# 4. Split the data
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# 5. Initialize and Train the Model
# We use LogisticRegression here for classification
log_model = LogisticRegression()
log_model.fit(X_train, y_train)

# 6. Make Predictions
y_pred = log_model.predict(X_test)

# 7. Evaluate
print("\nLogistic Regression Results:")
print(f"Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Logistic Regression Results:
Accuracy: 66.67%
Confusion Matrix:
[[0 1]
 [0 2]]
```

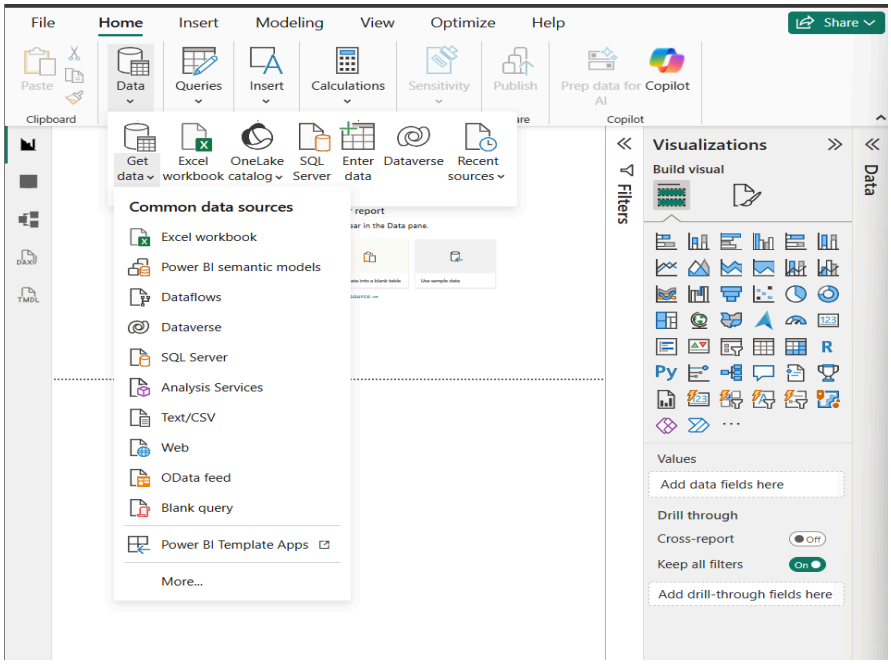
**The Explanation:**

- **Why `LogisticRegression()` instead of Linear?** If we used a straight line here, it might predict a churn value of `1.5` or `-0.2`, which makes no sense for a Yes/No question. Logistic regression squashes the output into an S-curve, giving us a probability between 0 and 1. If the probability is over 0.5, it predicts "1" (Churn). If it is under 0.5, it predicts "0" (Stay).
- **Why `confusion_matrix`?** This is a powerful BI tool. It doesn't just tell you the overall accuracy; it breaks down exactly *how* the model got things right or wrong (e.g., how many false positives versus false negatives).

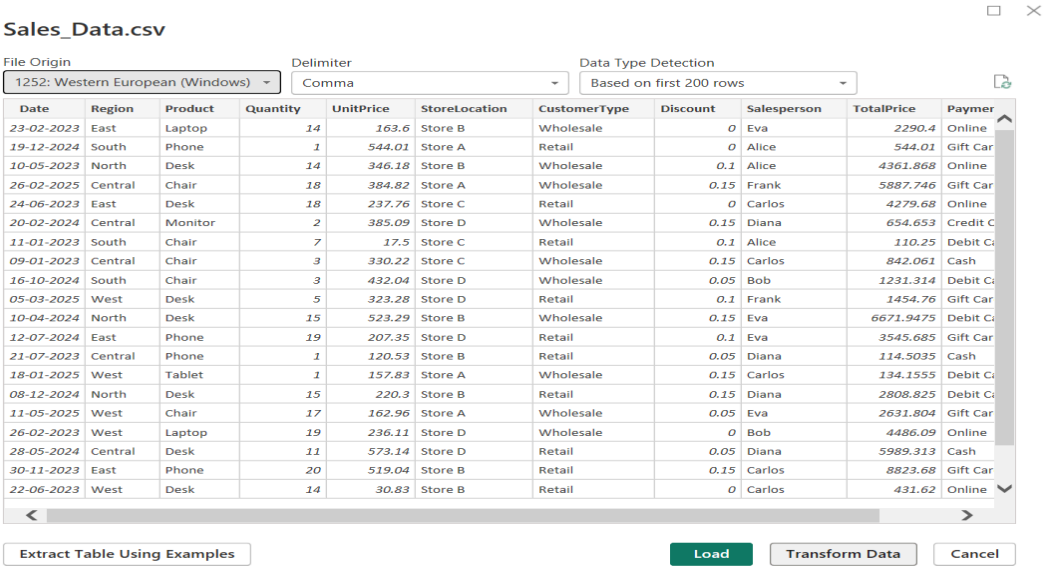
| | |
|---|---|
| 6. | Create the ETL map for the above and setup the schedule for execution |

**Part A: The ETL Map (Using Power Query)**

1. **Extract:** Open Power BI Desktop. Click **Get Data** > **Text/CSV** and select your `Sales_Data.csv` file.

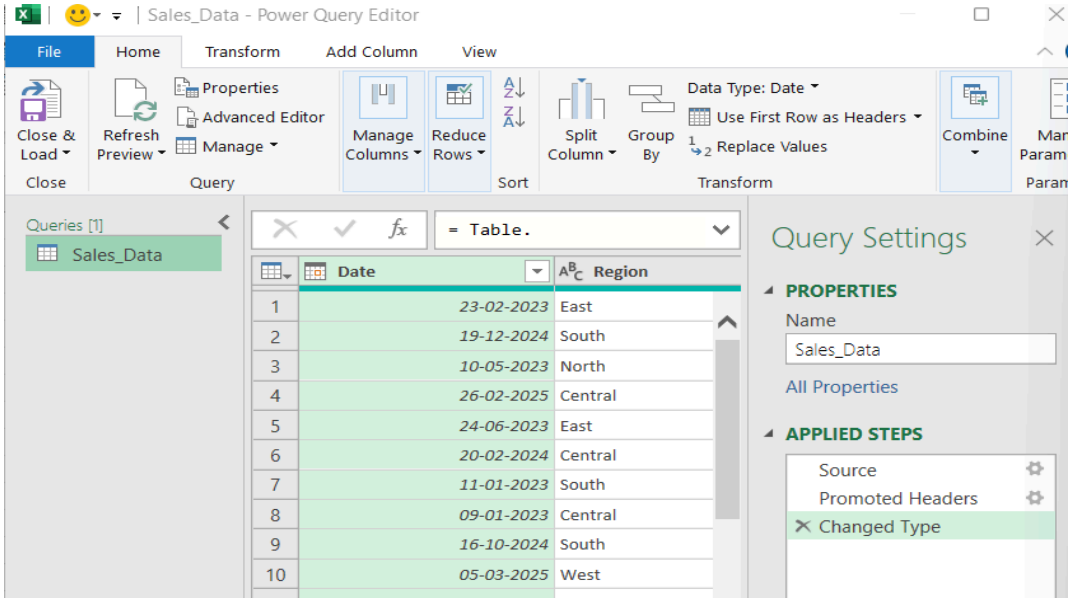



2. **Transform (Creating the Map):** Instead of clicking "Load", click **Transform Data**. This opens the **Power Query Editor**.

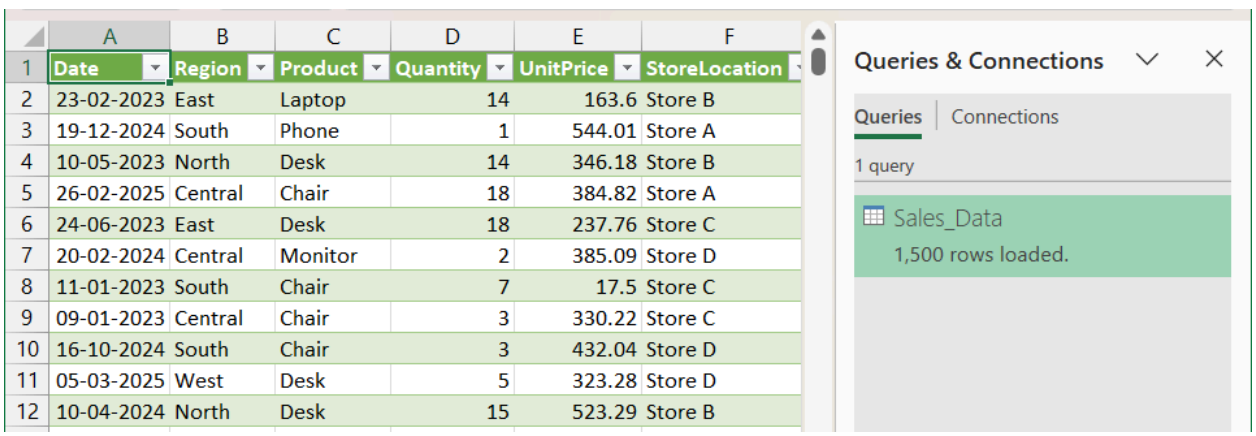



| Date | Region | Product | Quantity | UnitPrice | StoreLocation | CustomerType | Discount | Salesperson | TotalPrice | Paymer |
|---|---|---|---|---|---|---|---|---|---|---|
| 23-02-2023 | East | Laptop | 14 | 163.6 | Store B | Wholesale | 0 | Eva | 2290.4 | Online |
| 19-12-2024 | South | Phone | 1 | 544.01 | Store A | Retail | 0 | Alice | 544.01 | Gift Car |
| 10-05-2023 | North | Desk | 14 | 346.18 | Store B | Wholesale | 0.1 | Alice | 4361.868 | Online |
| 26-02-2025 | Central | Chair | 18 | 384.82 | Store A | Wholesale | 0.15 | Frank | 5887.746 | Gift Car |
| 24-06-2023 | East | Desk | 18 | 237.76 | Store C | Retail | 0 | Carlos | 4279.68 | Online |
| 20-02-2024 | Central | Monitor | 2 | 385.09 | Store D | Wholesale | 0.15 | Diana | 654.653 | Credit C |
| 11-01-2023 | South | Chair | 7 | 17.5 | Store C | Retail | 0.1 | Alice | 110.25 | Debit C |
| 09-01-2023 | Central | Chair | 3 | 330.22 | Store C | Wholesale | 0.15 | Carlos | 842.061 | Cash |
| 16-10-2024 | South | Chair | 3 | 432.04 | Store D | Wholesale | 0.05 | Bob | 1231.314 | Debit C |
| 05-03-2025 | West | Desk | 5 | 323.28 | Store D | Retail | 0.1 | Frank | 1454.76 | Gift Car |
| 10-04-2024 | North | Desk | 15 | 523.29 | Store B | Wholesale | 0.15 | Eva | 6671.9475 | Debit C |
| 12-07-2024 | East | Phone | 19 | 207.35 | Store D | Retail | 0.1 | Eva | 3545.685 | Gift Car |
| 21-07-2023 | Central | Phone | 1 | 120.53 | Store B | Retail | 0.05 | Diana | 114.5035 | Cash |
| 18-01-2025 | West | Tablet | 1 | 157.83 | Store A | Wholesale | 0.15 | Carlos | 134.1555 | Debit C |
| 08-12-2024 | North | Desk | 15 | 220.3 | Store B | Retail | 0.15 | Diana | 2808.825 | Debit C |
| 11-05-2025 | West | Chair | 17 | 162.96 | Store A | Wholesale | 0.05 | Eva | 2631.804 | Gift Car |
| 26-02-2023 | West | Laptop | 19 | 236.11 | Store D | Wholesale | 0 | Bob | 4486.09 | Online |
| 28-05-2024 | Central | Desk | 11 | 573.14 | Store D | Retail | 0.05 | Diana | 5989.313 | Cash |
| 30-11-2023 | East | Phone | 20 | 519.04 | Store B | Retail | 0.15 | Carlos | 8823.68 | Gift Car |
| 22-06-2023 | West | Desk | 14 | 30.83 | Store B | Retail | 0 | Carlos | 431.62 | Online |

- Look at the right side of the screen for the **"Applied Steps"** pane. **This is your ETL Map!** * Every action you take is recorded here sequentially. If you rename `StoreLocation` to `Branch`, or change `TotalPrice` to a Currency data type, a new step appears in the list.




3. **Load:** Once your data is clean, click **Close & Apply** at the top left. The data is now loaded into the Power BI Data Model.

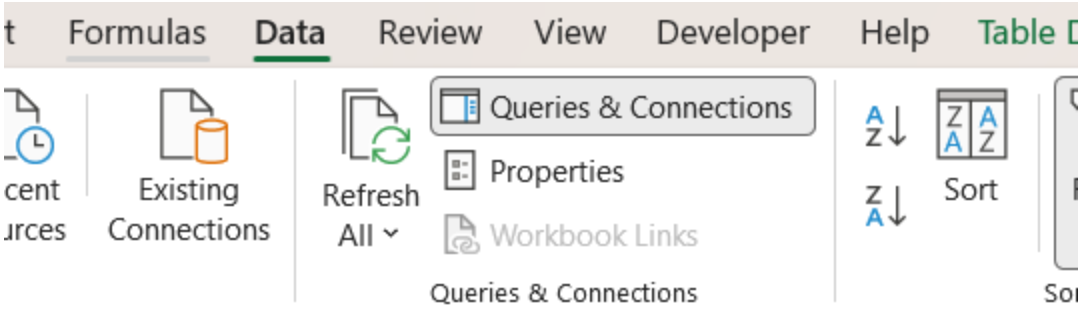


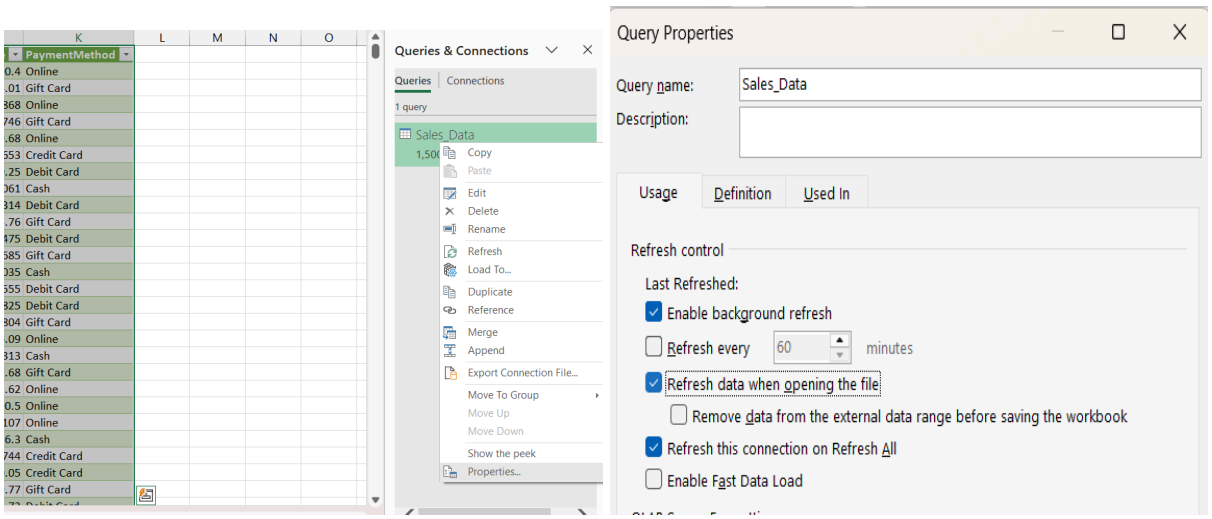
| | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| 1 | Date | Region | Product | Quantity | UnitPrice | StoreLocation |
| 2 | 23-02-2023 | East | Laptop | 14 | 163.6 | Store B |
| 3 | 19-12-2024 | South | Phone | 1 | 544.01 | Store A |
| 4 | 10-05-2023 | North | Desk | 14 | 346.18 | Store B |
| 5 | 26-02-2025 | Central | Chair | 18 | 384.82 | Store A |
| 6 | 24-06-2023 | East | Desk | 18 | 237.76 | Store C |
| 7 | 20-02-2024 | Central | Monitor | 2 | 385.09 | Store D |
| 8 | 11-01-2023 | South | Chair | 7 | 17.5 | Store C |
| 9 | 09-01-2023 | Central | Chair | 3 | 330.22 | Store C |
| 10 | 16-10-2024 | South | Chair | 3 | 432.04 | Store D |
| 11 | 05-03-2025 | West | Desk | 5 | 323.28 | Store D |
| 12 | 10-04-2024 | North | Desk | 15 | 523.29 | Store B |



**Part B: The Scheduling Catch** Excel does not have a built-in "run at 2 AM" server schedule like Power BI or SSIS. Your options are limited to:

- **The "Open File" Schedule:** In Excel, go to Data > Queries & Connections > right-click your query > Properties. You can check a box that says **"Refresh data when opening the file**




| | |
|---|---|
| 8. | Perform the Linear regression on the given data warehouse data Python/R. |

| | |
|---|---|
| 19. | Show prediction Using Linear Regression |

Before writing the code, let's define our **Objective**. Linear Regression is used to predict a *continuous numerical value* by finding the mathematical relationship between variables. The classic formula we are trying to map is the equation of a straight line:

$$y = \beta_0 + \beta_1 x + \epsilon$$

(Where $y$ is what we want to predict, $x$ is our input, $\beta_0$ is the intercept, $\beta_1$ is the slope, and $\epsilon$ is the error term).

**The Business Scenario:** We want to predict the `TotalPrice` (our Target, $y$) based purely on the `Quantity` of items a customer buys (our Feature, $x$).

Here is the Python implementation using your real dataset.

## The Python Implementation

You would run this in your Jupyter Notebook or Python IDE, making sure the `Sales_Data.csv` file is in the same folder as your script.

```
# 1. Import necessary libraries

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import r2_score, mean_squared_error

# 2. Load the real dataset

df = pd.read_csv('Sales_Data.csv')

# 3. Define Features (X) and Target (y)

# We use double brackets for X because scikit-learn expects a 2D array for features

X = df[['Quantity']]

y = df['TotalPrice']

# 4. Split the data into Training and Testing sets (80% train, 20% test)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 5. Initialize and Train the Linear Regression Model

model = LinearRegression()

model.fit(X_train, y_train)

# 6. Make Predictions on the unseen test data
```

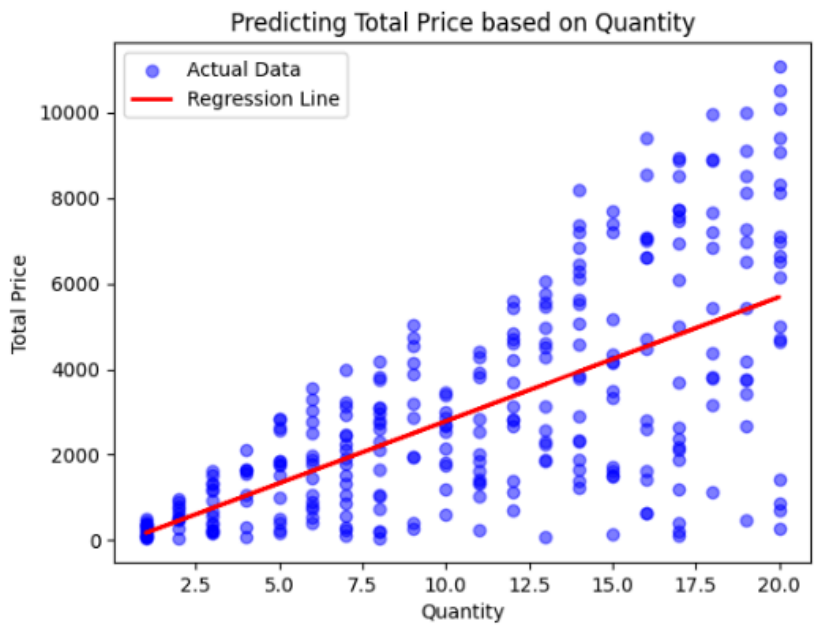
```
y_pred = model.predict(X_test)

# 7. Evaluate the Model's performance

r2 = r2_score(y_test, y_pred)

print(f"R-squared Score: {r2:.2f}")

# 8. Visualize the result (Crucial for BI presentations!)

plt.scatter(X_test, y_test, color='blue', alpha=0.5, label='Actual Data')

plt.plot(X_test, y_pred, color='red', linewidth=2, label='Regression Line')

plt.title('Predicting Total Price based on Quantity')

plt.xlabel('Quantity')

plt.ylabel('Total Price')

plt.legend()

plt.show()
```

```
#testing.
print(model.predict([[14]]))
```

[3] ✓ 0.0s

```
[3943.82776166]
c:\Users\saksh\AppData\Local\Programs\P
  warnings.warn(
```

```
R-squared Score: 0.45
```




## Step-by-Step Explanation: The "Why"

- **Step 2 (Loading Data):** Instead of typing out fake arrays, `pd.read_csv()` ingests your entire sales data into a Pandas DataFrame, automatically detecting the columns.
- **Step 3 (X and y):** In machine learning, `X` always represents your inputs (independent variables) and `y` represents the output you are trying to guess (dependent variable). We are asking the algorithm: "If I tell you the `Quantity`, can you guess the `TotalPrice`?"
- **Step 4 (Train/Test Split):** We hold back **20%** of the rows from `Sales_Data.csv`. The model will learn the relationship using 80% of the data, and then we will test it on the remaining 20% to see if it actually learned the pattern or just memorized the answers.
- **Step 5 (Fit):** This is where the heavy mathematical lifting happens. `.fit()` calculates that exact slope and intercept for the line of best fit.
- **Step 7 (R-squared Score):** This is your main grading metric. It returns a value between 0 and 1. If it returns `0.85`, it means 85% of the variance in `TotalPrice` can be perfectly explained by the `Quantity` purchased.
- **Step 8 (Visualization):** In Business Intelligence, you can't just hand a stakeholder a block of code. By plotting the actual data points (blue dots) against our model's predictions (the straight red line), the business can visually verify if the model makes sense.

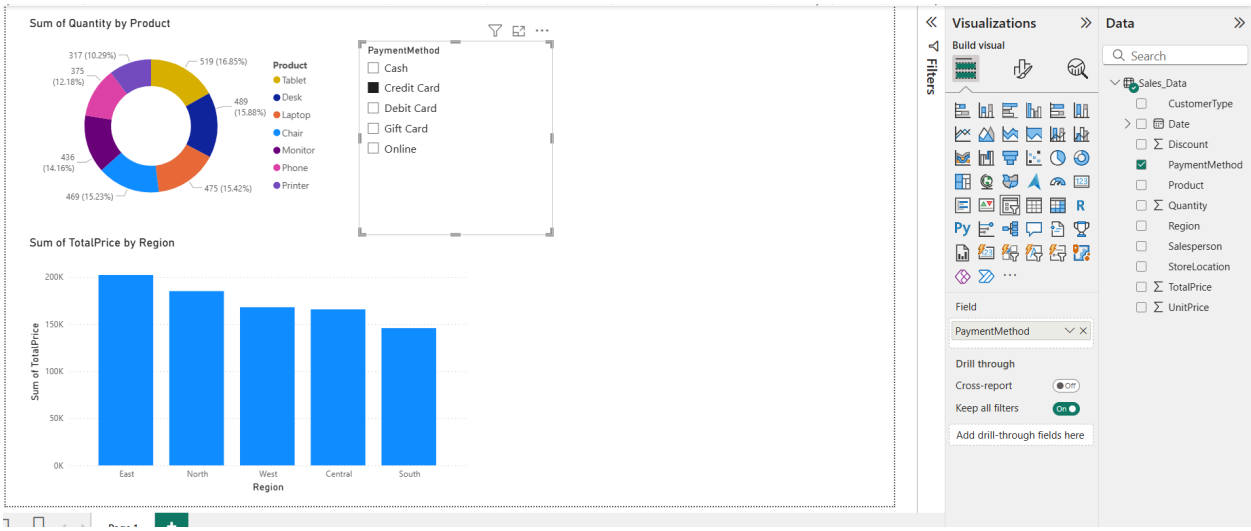
| | |
|---|---|
| 9. | **(A)** Perform data visualization using PowerBI on any sales data.<br>**(B)** Import the data warehouse data in Microsoft Excel and create the Pivot table and Pivot Chart. |

## Part A: Perform data visualization using PowerBI on any sales data

**The Objective:** Transform raw tabular data into an interactive, visual dashboard that tells a story about business performance at a glance. We will use your Sales_Data.csv file for this.

**Steps & Configuration:**

1. **Load the Data:** Open Power BI Desktop. Click **Get Data** > **Text/CSV** and select Sales_Data.csv. Click **Load** (or *Transform Data* if you need to clean it first, as we discussed in Q6).
2. **Build Chart 1 (Sales by Region):** * Go to the **Visualizations** pane on the right and click the **Clustered Column Chart** icon.
    - From the **Data** pane, drag Region into the *X-axis* field.
    - Drag TotalPrice into the *Y-axis* field. Power BI will automatically sum it up.
3. **Build Chart 2 (Product Popularity):**
    - Click a blank space on your canvas, then click the **Donut Chart** or **Pie Chart** icon.
    - Drag Product to the *Legend* field and Quantity to the *Values* field.
4. **Add a Slicer (Interactive Filter):**
    - Click a blank space, then click the **Slicer** icon (it looks like a funnel with a table).
    - Drag PaymentMethod or StoreLocation into the slicer field.




**The Explanation:**

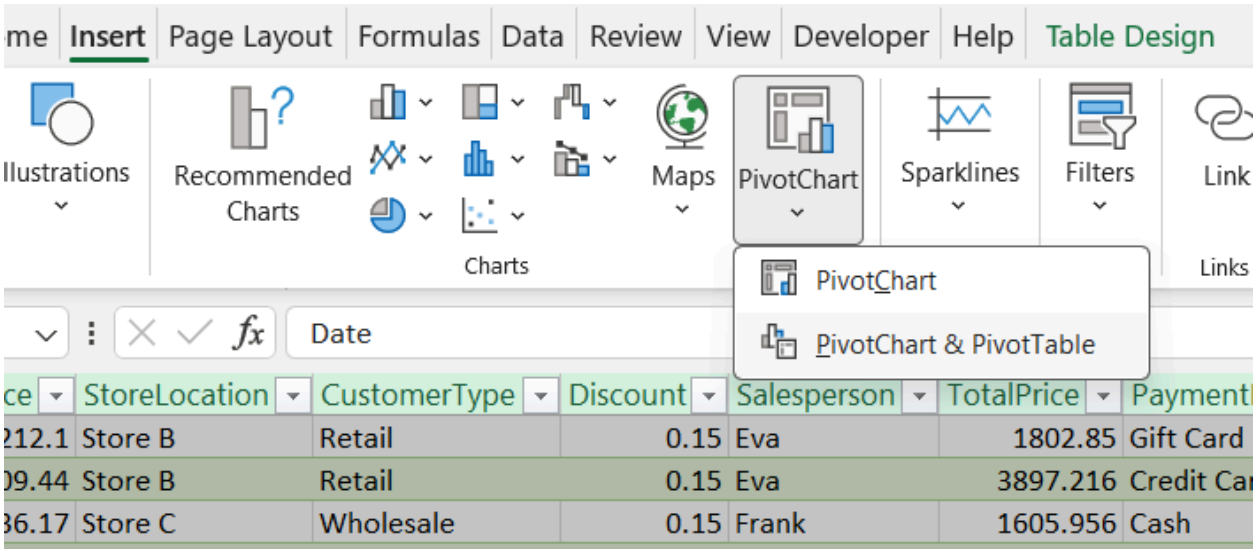
Why use Power BI? Power BI dashboards are highly interactive. If your professor clicks on "Laptop" in your Donut chart, the Column chart will instantly filter to only show Laptop sales across the different regions. This interactivity allows stakeholders to answer their own questions without needing to write a single line of SQL.

## Part B: Import the data warehouse data in Microsoft Excel and create the Pivot table and Pivot Chart

**The Objective:** We are going to take the raw, row-by-row transactional data from the CSV and aggregate it to answer a specific business question: **Which regions are generating the most revenue, and what products are driving those sales?**

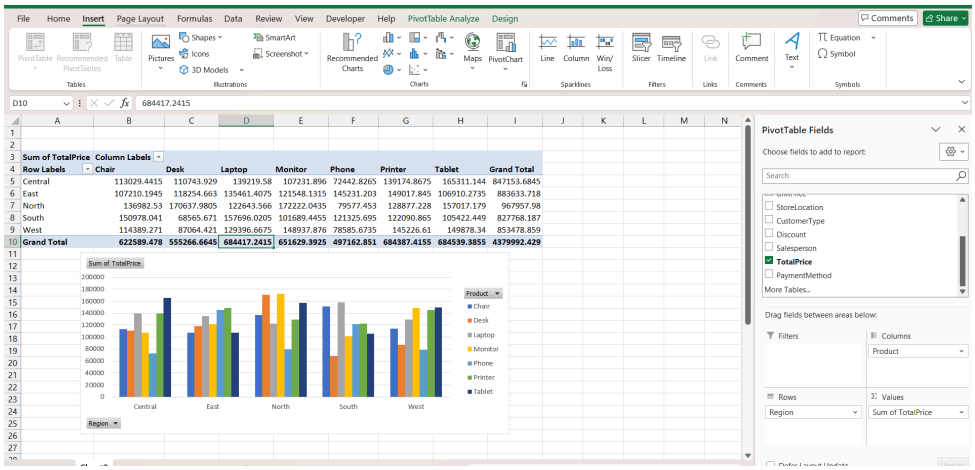
### The Steps: CSV to Pivot Chart in Excel

1. **Open the Data:**
   - Open Microsoft Excel.
   - Go to **File** > **Open** and browse for your `Sales_Data.csv` file. (You might need to change the file type dropdown in the open window to "All Files" or "Text Files" to see it). Or **just import the csv file data by going to data tab.**
2. **Insert the Pivot Table:**
   - Click anywhere inside your data (e.g., cell A1).
   - Press **Ctrl + A** to select the entire dataset.
   - On the top ribbon, go to the **Insert** tab and click **PivotTable**.
   - In the dialog box, ensure "New Worksheet" is selected and click **OK**.




3. **Configure the Pivot Table (The Analysis):**
   - You will now see a blank grid on the left and a **PivotTable Fields** pane on the right.
   - Drag the `Region` field down into the **Rows** area. You will see your rows instantly collapse into just Central, East, North, South, and West.
   - Drag the `Product` field down into the **Columns** area.
   - Drag the `TotalPrice` field down into the **Values** area. Excel will automatically change this to "Sum of TotalPrice".

4. **Create the Pivot Chart (The Visualization):**
    - Click anywhere inside the Pivot Table you just built.
    - Go to the **Insert** tab on the ribbon.
    - Click **PivotChart** (or **Recommended Charts**).
    - Select a **Clustered Column** chart and click **OK**.




**The Explanation:** Why do we still use Excel if Power BI is so visual? Because finance and accounting teams often prefer looking at grids of numbers rather than charts. Connecting Excel directly to the Data Warehouse ensures that everyone in the company is looking at a "single version of the truth." If the database updates overnight, the user just clicks "Refresh" in Excel, and their Pivot Table updates instantly.

| | |
|---|---|
| 10. | Write a Python program to read data from a CSV file, perform simple analysis, and generate insights. |

This is brilliant! Thank you for sharing the actual dataset. It makes the practical so much more realistic.

Looking at the data you provided, we have some fantastic columns to work with: `hour_of_day`, `cash_type`, `money`, `coffee_name`, `Time_of_Day`, `Weekday`, `Month_name`, `Date`, and `Time`.

Let's tackle **Question 10: "Write a Python program to read data from a CSV file, perform simple analysis, and generate insights."** using this exact data.

## The Objective

We are going to perform **Exploratory Data Analysis (EDA)**. We will write a Python script that loads this CSV, calculates key business metrics, and prints out plain-English insights that the coffee shop owner can use to increase their profits.

## The Python Implementation

First, save the data you just pasted into a file named coffee_sales.csv in the exact same folder where you write your Python script.

```
# 1. Import the necessary library

import pandas as pd



# 2. Read data from the CSV file

# We use on_bad_lines='skip' just in case that long paragraph in row 1
causes parsing issues

df = pd.read_csv('Coffee_sales.csv', on_bad_lines='skip')



# 3. Perform Simple Analysis

# A. Calculate Total Revenue (summing the 'money' column)

total_revenue = df['money'].sum()



# B. Find the Most Popular Coffee (counting occurrences in 'coffee_name')

coffee_counts = df['coffee_name'].value_counts()

top_coffee = coffee_counts.index[0]

top_coffee_orders = coffee_counts.iloc[0]



# C. Find the Busiest Hour (counting occurrences in 'hour_of_day')

Apeak_hours = df['hour_of_day'].value_counts()

busiest_hour = peak_hours.index[0]



# D. Find the Busiest Day of the Week
```

```python
busiest_day = df['Weekday'].mode()[0]


# E. Compare Payment Methods (grouping by 'cash_type' and summing 'money')

payment_revenue = df.groupby('cash_type')['money'].sum()


# 4. Generate and Print Insights

print("="*40)

print("   COFFEE SHOP BUSINESS INSIGHTS")

print("="*40)

print(f"Insight 1: The shop generated a total revenue of ${total_revenue:.2f}.")

print(f"Insight 2: The most popular drink is the '{top_coffee}', ordered {top_coffee_orders} times.")

print(f"Insight 3: The absolute busiest time of day to schedule staff is {busiest_hour}:00.")

print(f"Insight 4: The busiest day of the week is {busiest_day}.")

print("\nInsight 5: Revenue breakdown by payment method:")

print(payment_revenue.to_string())

print("="*40)
```

## Step-by-Step Explanation for Your Viva/Exam:

If your professor asks you to explain *how* this works, here is how you break it down:

### 1. Importing `pandas`:

- **Why:** The `pandas` library is the industry standard for data manipulation in Python. It allows us to load a raw CSV into a "DataFrame" (which acts like a virtual SQL table or Excel spreadsheet inside our code) so we can run calculations on entire columns instantly.

**2. Reading the Data:**

- **Why:** pd.read_csv() is the command that connects our script to the external file. It automatically reads your headers (hour_of_day, money, coffee_name, etc.) so we can call them directly by their names.

**3. Performing the Analysis (The Math):**

- **df['money'].sum():** We isolate the money column and add up every single row to calculate the gross revenue.
- **df['coffee_name'].value_counts():** This is a very powerful function. It looks at the coffee_name column, identifies every unique drink (Latte, Americano, Cortado, etc.), counts how many times they appear, and sorts them from highest to lowest.
- **df['Weekday'].mode()[0]:** The mode() function finds the most frequently occurring value in a column. This instantly tells us which day of the week brings in the most foot traffic.
- **df.groupby('cash_type')['money'].sum():** This acts exactly like an Excel Pivot Table! We are telling Python to group the rows by cash_type (like 'card') and then sum up the money for those specific groups.

**4. Generating Insights:**

- **Why print it this way?** In Business Intelligence, executives do not want to see raw arrays or code. By using Python f-strings (e.g., f"Total revenue is ${total_revenue}"), we dynamically inject our mathematical variables into readable, actionable English sentences.

```
========================================
   COFFEE SHOP BUSINESS INSIGHTS
========================================
Insight 1: The shop generated a total revenue of $112245.58.
Insight 2: The most popular drink is the 'Americano with Milk', ordered 809 times.
Insight 3: The absolute busiest time of day to schedule staff is 10:00.
Insight 4: The busiest day of the week is Tue.

Insight 5: Revenue breakdown by payment method:
cash_type
card    112245.58
========================================
```

| | |
|---|---|
| 12. | Perform the data clustering using clustering algorithm in Python/R Programming. |

Clustering is **Unsupervised Learning**. We are not trying to predict a specific answer. Instead, we are asking the AI: *"Look at the hour_of_day and the money spent. Do you see any natural*

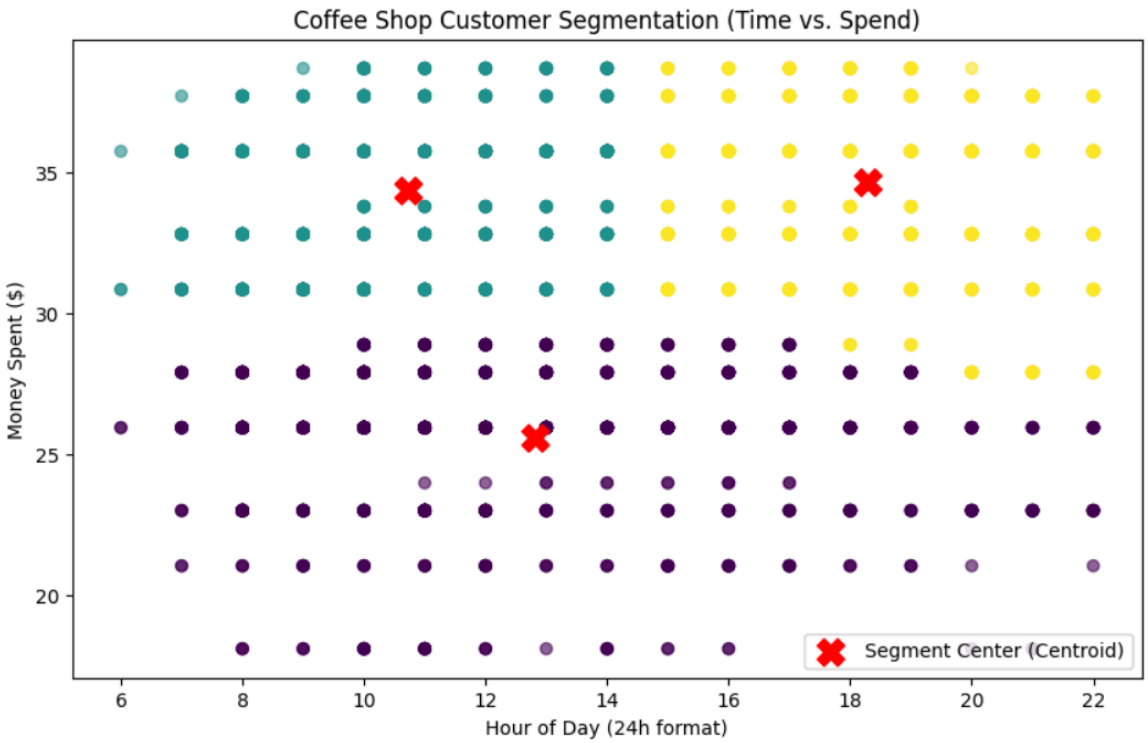
*groupings or hidden behaviors in this data?"* For a coffee shop owner, this reveals actionable business segments (e.g., identifying a distinct cluster of "late-night big spenders" versus "morning quick-coffee buyers").

## The Python Implementation

```
# 1. Import necessary libraries

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

# 2. Load the dataset (skipping bad lines just in case of formatting issues)

df = pd.read_csv('coffee_sales.csv', on_bad_lines='skip')

# 3. Select the Features for Clustering

# We want to group transactions based on Time (hour) and Value (money)

# We use dropna() to ensure no blank cells break our math

X = df[['hour_of_day', 'money']].dropna()

# 4. Initialize the K-Means Model

# We are asking the algorithm to find exactly 3 distinct customer segments

kmeans = KMeans(n_clusters=3, random_state=42)

# 5. Train the Model and Assign Clusters

# fit_predict calculates the distances and immediately tags each row with a cluster (0, 1, or 2)

clusters = kmeans.fit_predict(X)

# Add these new cluster labels back to a copy of our data so we can see them

df_clustered = X.copy()
```

```
df_clustered['Customer_Segment'] = clusters

# 6. Visualize the Results (Crucial for BI reporting)

plt.figure(figsize=(10, 6))

# Plot the individual customer transactions, colored by their assigned segment

plt.scatter(df_clustered['hour_of_day'], df_clustered['money'],

            c=df_clustered['Customer_Segment'], cmap='viridis', alpha=0.6)

# Plot the "Centroids" (the mathematical center of each group) in red

plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],

            s=200, c='red', marker='X', label='Segment Center (Centroid)')

plt.title('Coffee Shop Customer Segmentation (Time vs. Spend)')

plt.xlabel('Hour of Day (24h format)')

plt.ylabel('Money Spent ($)')

plt.legend()

plt.show()
```

...




## Step-by-Step Explanation for your Viva/Exam

If your examiner asks you to explain the underlying logic of this code, focus on these key points:

**1. Feature Selection (`X`):**

- **Why:** The algorithm needs numerical axes to plot data on a virtual graph. By giving it `hour_of_day` and `money`, it treats every coffee order as an $(x, y)$ coordinate.

**2. The K-Means Math (`KMeans(n_clusters=3)`):**

- **Why:** The "K" stands for the number of groups we want. The algorithm mathematically calculates the straight-line spatial distance between all the data points using the Euclidean distance formula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- It groups the data points that are mathematically closest to each other.

**3. The Centroids (`cluster_centers_`):**

- **Why:** The red 'X' marks we plotted are the literal "centers of gravity" for each segment. The algorithm starts by dropping 3 random points on the graph, measures which

transactions are closest to them, and iteratively adjusts the points until they sit perfectly in the center of the densest groups of data.

**4. The Business Value:**

- **Why visualize it?** When the plot renders, the business owner might physically see a dense cluster of dots at `hour_of_day = 10` with `money = 38.7`. This tells the BI team exactly when the highest-value transactions naturally cluster together without anyone having to manually sort through the CSV rows.

| | |
|---|---|
| 11. | **(A)** Perform the Extraction Transformation and Loading (ETL) process to construct the database in the Sql server.<br>**(B)** Apply the what – if Analysis for data visualization. Design and generate necessary reports based on the data warehouse data. |

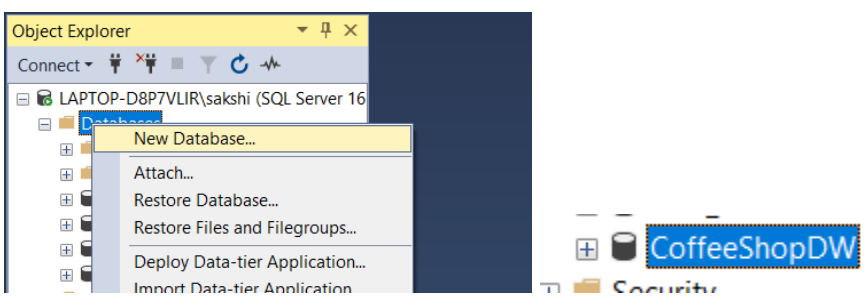
## Part 11(A): Perform the ETL process to construct the database in SQL Server

**The Objective:** Take the raw, unstructured `coffee_sales.csv` file, clean it, and load it into a structured SQL Server database table so it acts as our "Data Warehouse" for reporting.

We will use **SQL Server Integration Services (SSIS)**.

**Steps & Commands:**

1. **Prepare the Destination (SQL Server):**
   - Open **SQL Server Management Studio (SSMS)**.
   - Right-click "Databases" -> **New Database**. Name it `CoffeeShopDW`.



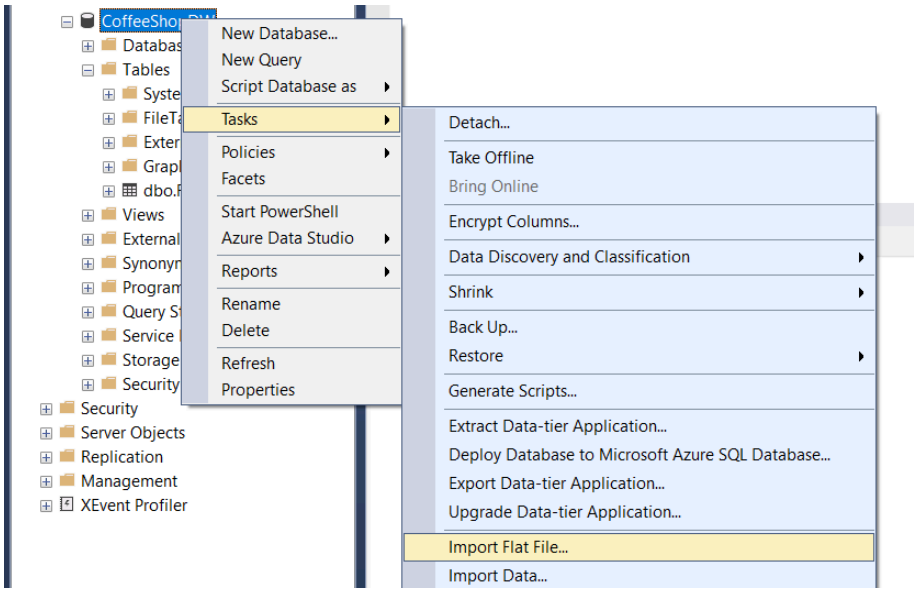

   - Open a New Query and create your destination table:

**CREATE TABLE Fact_CoffeeSales (**

```
    TransactionID INT IDENTITY(1,1) PRIMARY KEY,
    HourOfDay INT,
    CashType VARCHAR(50),
    MoneySpent DECIMAL(10,2),
    CoffeeName VARCHAR(100),
    WeekdayName VARCHAR(20)
);
```

## Step 2: Launch the ETL Wizard

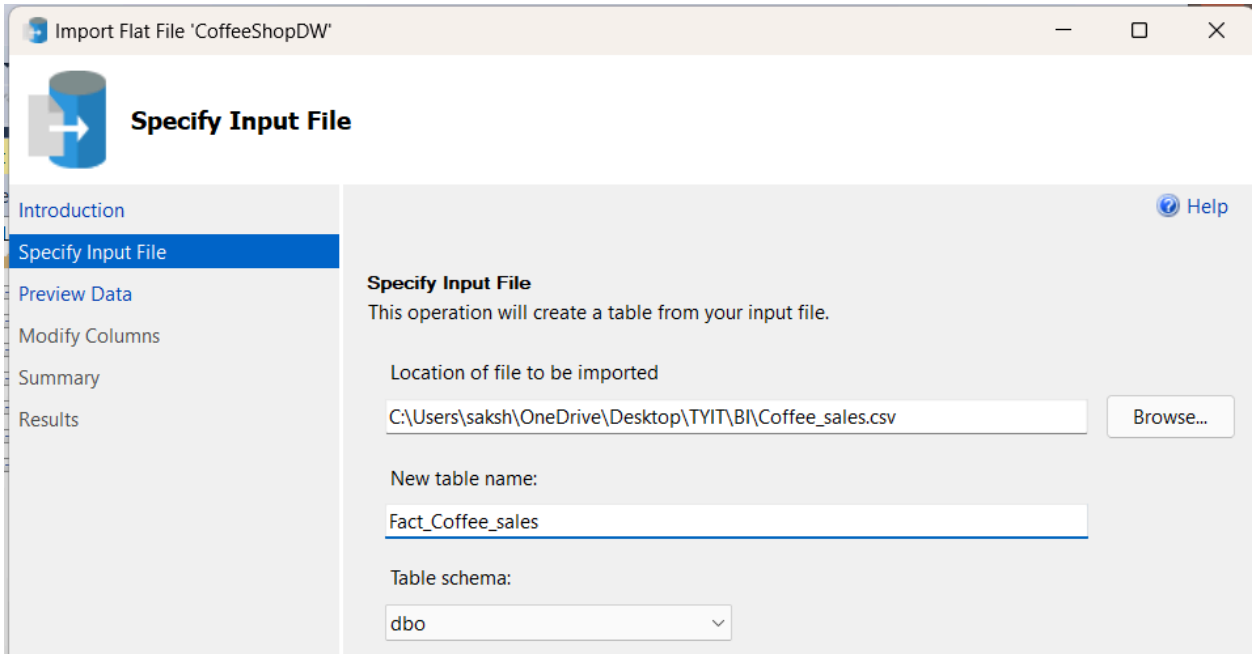
1. Right-click on your newly created `CoffeeShopDW` database.
2. Go down to Tasks.
3. Select Import Flat File... (If you are on an older version of SSMS, it might just say Import Data...).




## Step 3: Extract & Transform via the Wizard

A new window will pop up. This wizard is your ETL pipeline!

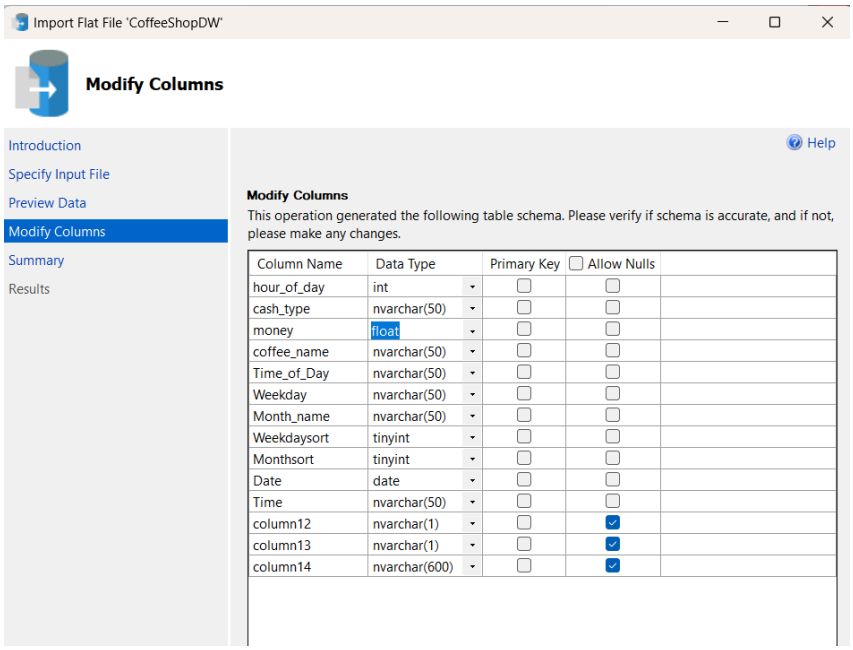
1. Specify Input File (Extract): Click Browse and find your `coffee_sales.csv` file. Give your new table a name, like `Fact_CoffeeSales`. Click Next.




Preview Data: It will show you a snapshot of your CSV to make sure it looks right. Click Next.
Modify Columns (Transform): This is the crucial step! The wizard tries to guess the data types, but you must check them.

- Make sure `hour_of_day` is set to an integer type (like `smallint` or `int`).
- Make sure `money` is set to a decimal type (like `decimal`, `float`, or `real`).
- *Why?* This fulfills the "Transformation" part of the practical by ensuring clean data formatting.

Click Next.



## Step 4: Load

1. You will reach a Summary page. Click Finish.
2. The wizard will run. You will see a green success checkmark as it automatically creates the table and loads every row from your CSV into the SQL Server database.

Results | Messages

| | hour_of_day | cash_type | money | coffee_name | Time_of_Day | Weekday | Month_name | Weekdaysort | Monthsort | Date | Time | column12 | column13 | column14 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 3 | 12 | card | 38.7000007629395 | Hot Chocolate | Afternoon | Fri | Mar | 5 | 3 | 2024-03-01 | 20:18.1 | NULL | NULL | NULL |
| 4 | 13 | card | 28.8999996185303 | Americano | Afternoon | Fri | Mar | 5 | 3 | 2024-03-01 | 46:33.0 | NULL | NULL | NULL |
| 5 | 13 | card | 38.7000007629395 | Latte | Afternoon | Fri | Mar | 5 | 3 | 2024-03-01 | 48:14.6 | NULL | NULL | NULL |
| 6 | 15 | card | 33.7999992370605 | Americano with Milk | Afternoon | Fri | Mar | 5 | 3 | 2024-03-01 | 39:47.7 | NULL | NULL | NULL |
| 7 | 16 | card | 38.7000007629395 | Hot Chocolate | Afternoon | Fri | Mar | 5 | 3 | 2024-03-01 | 19:02.8 | NULL | NULL | NULL |
| 8 | 18 | card | 33.7999992370605 | Americano with Milk | Night | Fri | Mar | 5 | 3 | 2024-03-01 | 39:03.6 | NULL | NULL | NULL |
| 9 | 19 | card | 38.7000007629395 | Cocoa | Night | Fri | Mar | 5 | 3 | 2024-03-01 | 22:01.8 | NULL | NULL | NULL |
| 10 | 19 | card | 33.7999992370605 | Americano with Milk | Night | Fri | Mar | 5 | 3 | 2024-03-01 | 23:15.9 | NULL | NULL | NULL |
| 11 | 19 | card | 33.7999992370605 | Americano with Milk | Night | Fri | Mar | 5 | 3 | 2024-03-01 | 29:17.4 | NULL | NULL | NULL |

**How to explain this to your examiner:** If your professor asks how you performed ETL without Visual Studio, you can confidently explain: *"I used the SQL Server Import Wizard. It **Extracted** the data from the flat CSV file, allowed me to **Transform** the schema by defining strict SQL data types for columns like 'money', and then **Loaded** the structured data into the 'CoffeeShopDW' database."*

## Part 11(B): Apply What-If Analysis for data visualization and generate reports

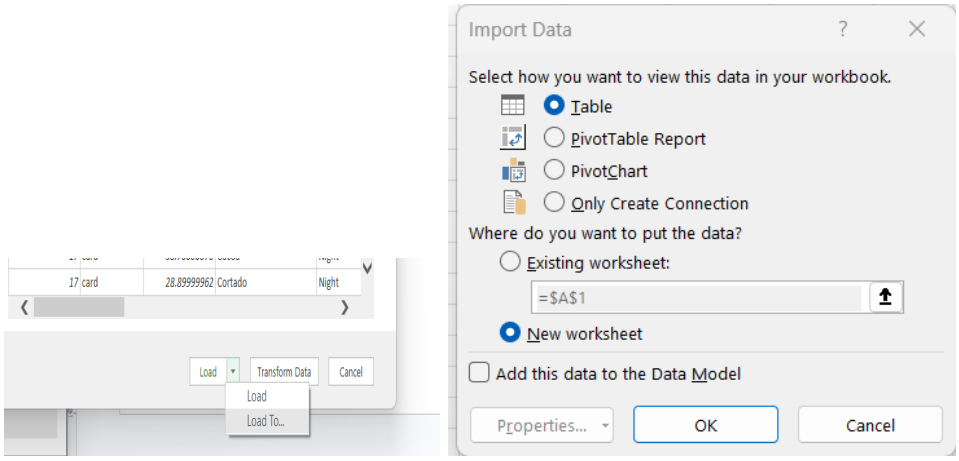
Since Question 11(B) specifically says "generate necessary reports based on the **data warehouse data**", we will connect Excel directly to the SQL Server database you just built in Part A, and then run a What-If scenario.

**The Business Scenario:** The coffee shop owner wants to know: *"If I increase my coffee prices by 5%, 10%, 15%, or 20% next month, what will my projected total revenue look like?"*

## Step 1: Connect Excel to your Data Warehouse

**The Objective:** Pull the structured data from SQL Server into Excel.

1. Open a blank Microsoft Excel workbook.
2. Go to the **Data** tab on the ribbon.
3. Click **Get Data** -> **From Database** -> **From SQL Server Database**.
4. Enter your server name (e.g., `localhost`) and your database name (`CoffeeShopDW`). Click **OK**.
5. Select your `Fact_CoffeeSales` table.
6. Click the arrow next to the **Load** button and choose **Load To...**
7. Select **Table** and click **OK**. Your database data will now appear in the Excel sheet.




## Step 2: Set Up the Base Calculations

**The Objective:** Calculate our current baseline revenue so we have something to manipulate.

Let's assume your imported data is in Columns A through F, and the `MoneySpent` column is Column D. Let's set up our dashboard on the right side of the screen (e.g., Column H).

1. In cell **M1**, type `Current Total Revenue`.
2. In cell **N1**, calculate the sum of your data warehouse sales: `=SUM(D:D)`
3. In cell **M2**, type `Proposed Price Increase %`.
4. In cell **N2**, type `0%` (This is our variable "What-If" cell).
5. In cell **M3**, type `Projected Revenue`.
6. In cell **N3**, enter the formula to calculate the new revenue: `=I1 + (I1 * I2)`

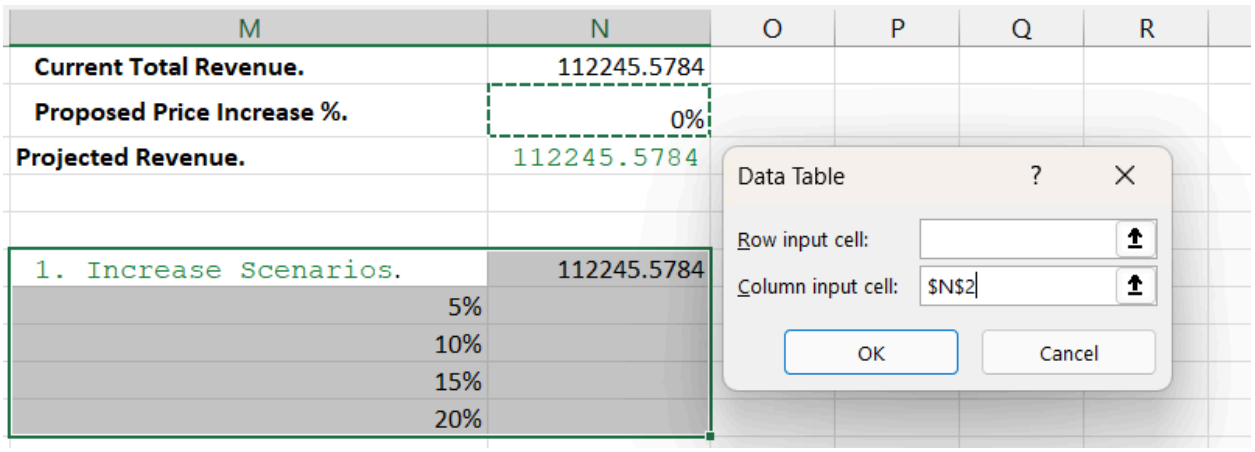
| M | N | O |
|---|---|---|
| **Current Total Revenue.** | 112245.6 | |
| **Proposed Price Increase %.** | 0% | |
| **Projected Revenue.** | =N1+(N1*N2) | |

## Step 3: Run the What-If Analysis (Data Table)

**The Objective:** Use Excel's built-in tool to automatically calculate multiple future scenarios at once.

1. Create a table for your scenarios. In cell **M6**, type `Increase Scenarios`.
2. In the cells below it (**M7 to M10**), type out your potential price increases: `5%`, `10%`, `15%`, `20%`.

3. In cell **N6** (the top of the column next to your percentages), link it to your projected revenue formula by typing: `=N3`
4. **Select the entire range** of your scenarios and the linked cell (Select from **M6 to N10**).
5. Go to the **Data** tab on the Excel ribbon.
6. Click on **What-If Analysis** and select **Data Table...**
7. Leave the *Row input cell* blank.
8. Click inside the **Column input cell** box, and click on cell **N2** (your `Proposed Price Increase %` cell).
9. Click **OK**.



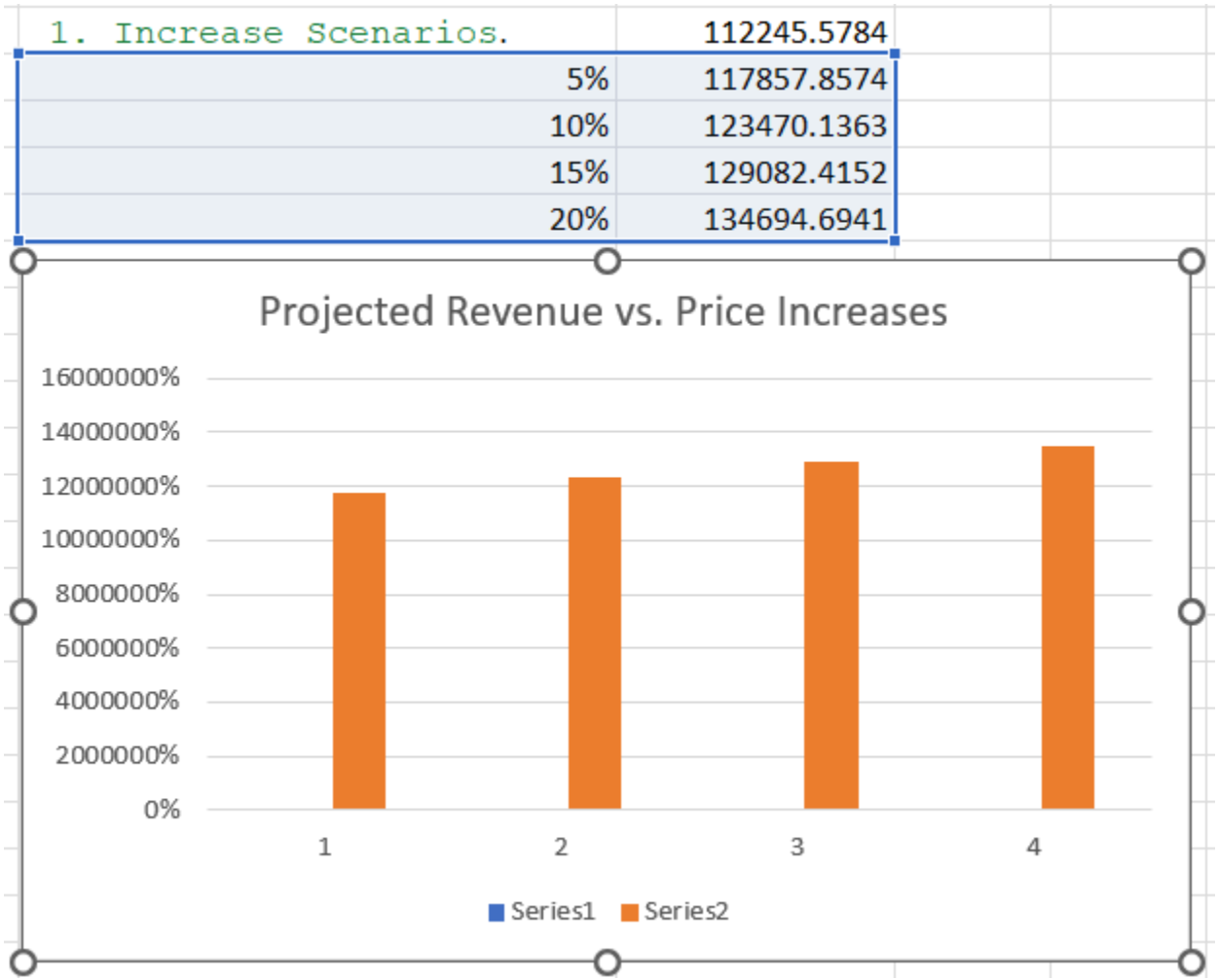

| M | N |
|---|---|
| **Current Total Revenue.** | 112245.5784 |
| **Proposed Price Increase %.** | 0% |
| **Projected Revenue.** | 112245.5784 |
| | |
| | |
| 1. Increase Scenarios. | 112245.5784 |
| 5% | 117857.8574 |
| 10% | 123470.1363 |
| 15% | 129082.4152 |
| 20% | 134694.6941 |

## Step 4: Data Visualization (Generate the Report)

**The Objective:** Translate the What-If numbers into a chart for the final report.

1. Highlight your newly generated Data Table results (cells **M7 to N10**).

2. Go to the **Insert** tab on the ribbon.
3. Click on the **Column Chart** icon and select a 2D Clustered Column.
4. Add a title to your chart, like "Projected Revenue vs. Price Increases".




| | |
|---|---|
| 14. | Perform data visualization using Python on any sales data Python/R. |

To address **Question 14**, "Perform data visualization using Python on any sales data", we will use your provided Sales_Data.csv.

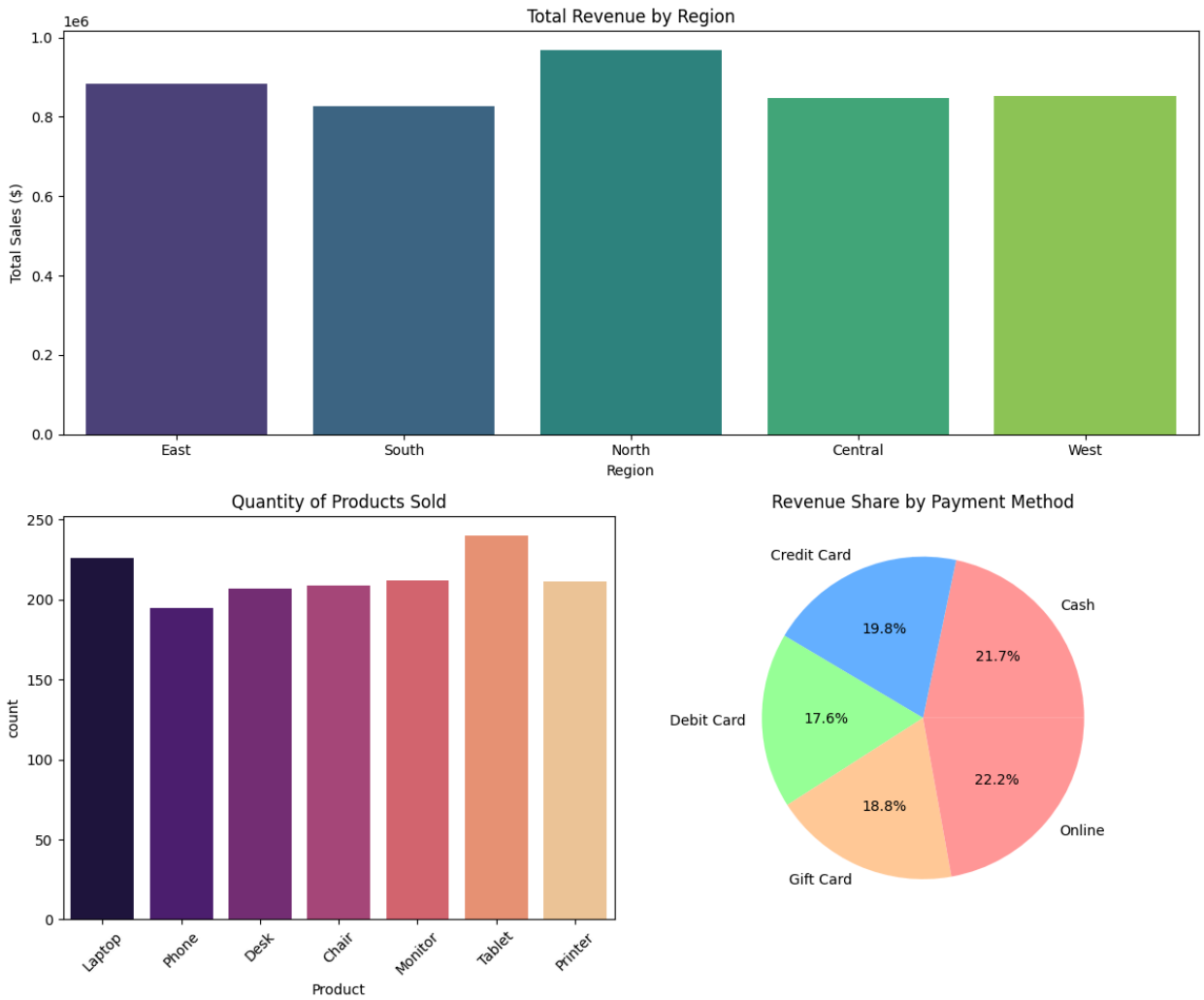
The objective of this practical is to move beyond simple numbers and create clear, visual representations that allow business managers to see trends in regions, products, and salesperson performance at a glance.

## The Python Implementation

For this task, we use the pandas library for data processing and matplotlib or seaborn for creating the actual charts.

```
import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

# 2. Load the Sales Data  # Make sure Sales_Data.csv is in the same folder as your script

df = pd.read_csv('Sales_Data.csv')

# 3. Data Visualization

plt.figure(figsize=(12, 10))

# --- Chart 1: Total Sales by Region ---# This shows which geographical area is performing best

plt.subplot(2, 1, 1) # First row of a 2x1 grid

sns.barplot(data=df, x='Region', y='TotalPrice', estimator=sum, ci=None, palette='viridis')

plt.title('Total Revenue by Region')

plt.ylabel('Total Sales ($)')

# --- Chart 2: Product Popularity (Quantity Sold) ---# This helps inventory managers see which items move fastest

plt.subplot(2, 2, 3) # Bottom left of a 2x2 grid

sns.countplot(data=df, x='Product', palette='magma')

plt.title('Quantity of Products Sold')

plt.xticks(rotation=45)

# --- Chart 3: Revenue by Payment Method  # This shows customer payment preferences

plt.subplot(2, 2, 4) # Bottom right of a 2x2 grid
```

```
df.groupby('PaymentMethod')['TotalPrice'].sum().plot(kind='pie', autopct='%1.1f%%', colors=['#ff9999','#66b3ff','#99ff99','#ffcc99'])

plt.title('Revenue Share by Payment Method')

plt.ylabel('') # Remove y-label for pie chart

# Adjust layout and show

plt.tight_layout()

plt.show()
```




## Step-by-Step Explanation: The "Why"

When presenting this to your examiner, focus on how these visualizations provide **Business Intelligence**:

1. **Revenue by Region (`sns.barplot`):**
    - **The Insight:** It aggregates the `TotalPrice` column based on the `Region` column.
    - **The BI Value:** If the "Central" region is significantly higher than "North," management can decide to shift marketing budgets or open new stores in the Central area.
2. **Product Sales (`sns.countplot`):**
    - **The Insight:** It counts how many times each `Product` (Laptop, Desk, Chair, etc.) appears in the transactions.
    - **The BI Value:** This is critical for Supply Chain management. If "Phones" are selling more than "Desks," the business knows to increase orders for Phones to avoid stock-outs.
3. **Revenue Share by Payment Method (`df.plot(kind='pie')`):**
    - **The Insight:** It shows the percentage of total money coming in through Online, Gift Card, Debit Card, etc.
    - **The BI Value:** If "Gift Card" sales are low, the marketing team might launch a promotion to encourage gift card purchases to increase customer loyalty.

## Teacher's Tip for the Exam:

Make sure you can explain the difference between a **Bar Chart** (used to compare totals between categories) and a **Pie Chart** (used to show parts of a whole). Examiners often ask why you chose one over the other!

| | |
|---|---|
| 15. | Write a Python program to read data from a CSV file containing columns such as 'Date', 'Product', 'Category', 'Sales', and 'Profit'. The program should perform simple analysis |

The task is to: "Write a Python program to read data from a CSV file containing columns such as 'Date', 'Product', 'Category', 'Sales', and 'Profit'. The program should perform simple analysis".

Since we are using your `Sales_Data.csv`, we will map your columns to the ones requested in the question:

- **'Product'** and **'Date'** exist in your data.
- **'Sales'** in the question corresponds to your **'TotalPrice'**.
- **'Profit'** in the question can be calculated in our script (e.g., assuming a profit margin).

## The Python Implementation

```python
import pandas as pd

# 2. Read the data

df = pd.read_csv('Sales_Data.csv')

# 3. Perform Simple Analysis

# A. Total Sales and Total Profit

total_sales = df['TotalPrice'].sum()

# Let's calculate a "Profit" column assuming profit is 20% of TotalPrice

df['Profit'] = df['TotalPrice'] * 0.20

total_profit = df['Profit'].sum()

# B. Analysis by Category (Mapping 'Product' as Category)

category_analysis = df.groupby('Product').agg({

    'TotalPrice': 'sum',

    'Profit': 'sum',

    'Quantity': 'sum'

})

# C. Find the Best Performing Product

best_product = category_analysis['TotalPrice'].idxmax()

# 4. Generate Insights

print("--- Sales and Profit Analysis Report ---")

print(f"Total Revenue Generated: ${total_sales:,.2f}")
```

```
print(f"Total Profit (Estimated 20%): ${total_profit:,.2f}")

print(f"The top-selling product is: {best_product}")

print("\nDetailed analysis per product category:")

print(category_analysis)
```

```
--- Sales and Profit Analysis Report ---
Total Revenue Generated: $4,379,992.43
Total Profit (Estimated 20%): $875,998.49
The top-selling product is: Tablet

Detailed analysis per product category:
          TotalPrice       Profit  Quantity
Product
Chair    622589.4780  124517.8956      2122
Desk     555266.6645  111053.3329      2026
Laptop   684417.2415  136883.4483      2408
Monitor  651629.3925  130325.8785      2177
Phone    497162.8510   99432.5702      1970
Printer  684387.4155  136877.4831      2336
Tablet   684539.3855  136907.8771      2577
```

## Step-by-Step Explanation: The "Why"

**1. Data Loading (`pd.read_csv`):**

- **Why:** This is the **Extraction** phase of the BI process. We are pulling raw data from a flat file into a structured format (a DataFrame) for processing.

**2. Aggregation (`.sum()` and `.groupby()`):**

- **Why:** Raw data doesn't provide insights. By grouping by 'Product' and summing 'TotalPrice', we perform **Descriptive Analytics**. It allows the business to see which products are the most valuable.
  **3. Calculated Fields (`df['Profit'] = ...`):**
- **Why:** In Business Intelligence, we often create **Calculated Measures**. If the raw data warehouse is missing a KPI like "Profit," we use Python to transform existing data into a new metric that stakeholders care about.

**4. Insights Generation:**

- **Why:** The end goal of any BI practical is to provide a conclusion. Identifying the `idxmax()` (the index of the maximum value) tells the manager exactly where to focus their inventory and marketing efforts.

| | |
|---|---|
| 16. | **(A)** Create the Data staging area for the selected database using SQL server.<br>**(B)** Import the legacy data from different sources such as ( Excel ) and load in the target system(SQL server) |

Before we write any code or click any buttons, let's understand the **Objective**.

What exactly is a **"Data Staging Area"**? Imagine you are cooking a big meal. You don't chop your vegetables directly over the boiling soup. You chop them on a cutting board first, remove the bad pieces, and *then* put them in the pot.

- The **Source (Excel)** is your grocery bag.
- The **Staging Area** is your cutting board (a temporary database where data lands exactly as it is).
- The **Data Warehouse** is the final pot of soup (clean, formatted, ready for reporting).

## Part 16(A): Create the Data Staging Area using SQL Server

**The Steps & Commands:**

1. Open **SQL Server Management Studio (SSMS)** and connect to your database engine.
2. Open a **New Query** window.
3. We will create a dedicated Staging Database and a temporary staging table to hold our Sales Data. Run the following SQL commands:

```
-- 1. Create a dedicated Staging Database

CREATE DATABASE SalesStagingArea;

GO

USE SalesStagingArea;

GO

-- 2. Create the Staging Table

-- Notice how every column is VARCHAR? This is a BI secret!

CREATE TABLE stg_LegacySales (
```

```
    Date VARCHAR(50),
    Region VARCHAR(50),
    Product VARCHAR(50),
    Quantity VARCHAR(50),
    UnitPrice VARCHAR(50),
    StoreLocation VARCHAR(50),
    CustomerType VARCHAR(50),
    Discount VARCHAR(50),
    Salesperson VARCHAR(50),
    TotalPrice VARCHAR(50),
    PaymentMethod VARCHAR(50)
);
```

**The Explanation (The "Why"):**

Why did I make every column `VARCHAR(50)` instead of using `INT` for Quantity or `DECIMAL` for Price?

Because this is *legacy data* coming from Excel! Excel files are notoriously messy. If someone accidentally typed "Ten" instead of "10" in the Quantity column, an `INT` data type would crash the entire import process. In a staging area, the golden rule is **"Extract as-is."** We pull the raw text in first, and we will convert it to proper numbers later when moving it from Staging to the final Data Warehouse.

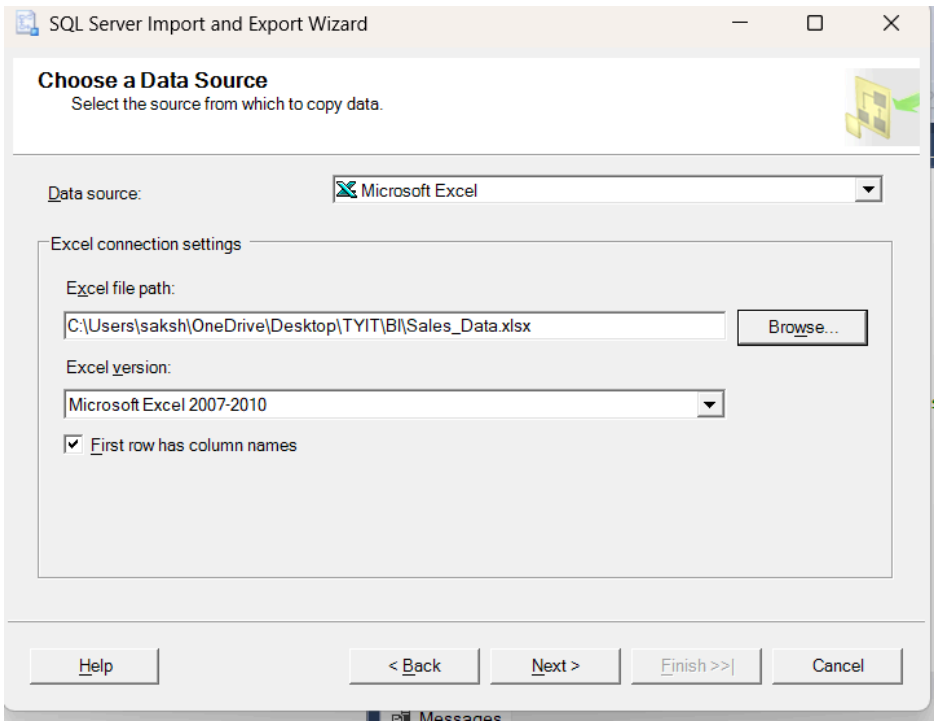
---

## Part 16(B): Import the Legacy Data from Excel to SQL Server

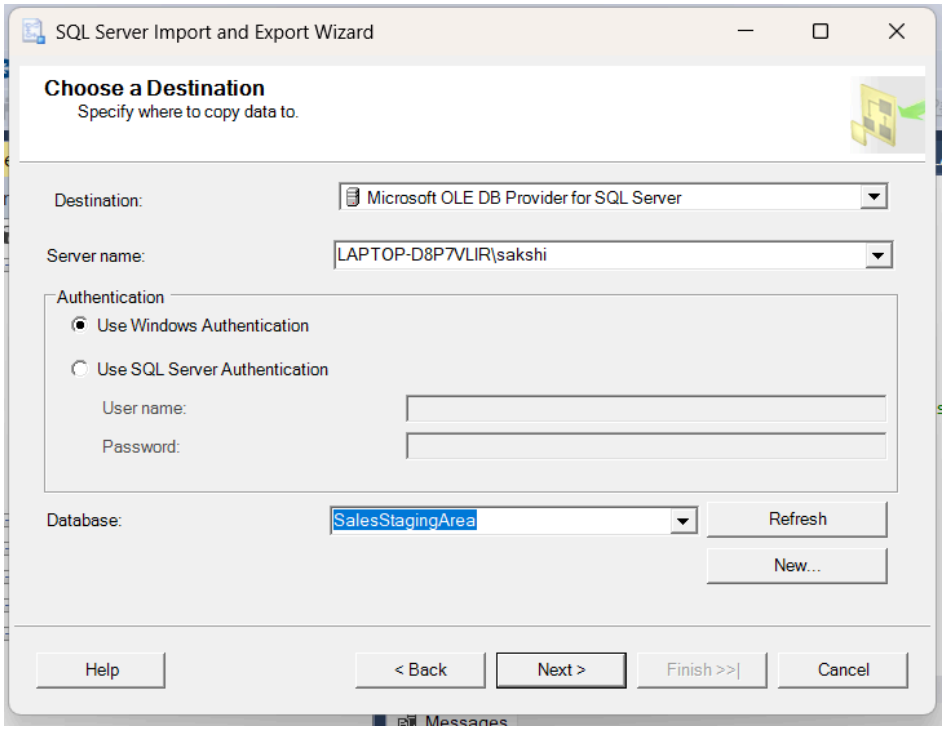
**The Objective:** Use the built-in wizard to pump the data from your Excel sheet into the staging table we just built. *(Note: If your sales data is currently a CSV, just open it in Excel and "Save As" an .xlsx workbook for this practical).*

**The Steps:**

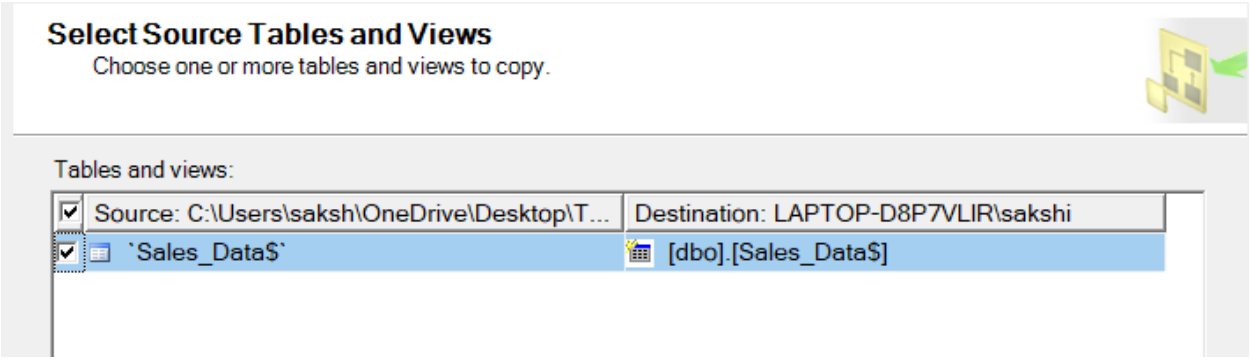
1. In SSMS, look at the **Object Explorer** on the left.
2. Right-click your newly created `SalesStagingArea` database.
3. Go down to **Tasks** > **Import Data...** This opens the SQL Server Import and Export Wizard.
4. **Choose a Data Source:**
    - Change the Data Source dropdown to **Microsoft Excel**.
    - Click **Browse** and select your Sales Data Excel file.
    - Make sure the box "First row has column names" is checked. Click Next.




5. **Choose a Destination:**
    - Change the Destination dropdown to **Microsoft OLE DB Provider for SQL Server** (or *SQL Server Native Client*).;
    - Enter your Server Name and ensure the Database is set to `SalesStagingArea`. Click Next.
6. **Specify Table Copy:**
    - Select "Copy data from one or more tables or views". Click Next.




7. **Select Source Tables and Views:**
    - Under "Source", check the box next to your Excel sheet (it usually says something like `Sheet1$`).
    - Under "Destination", click the dropdown and select the table we created in Part A: `[dbo].[stg_LegacySales]`.
8. Click **Next**, then **Finish**.




You will see a loading screen as rows are successfully transferred into your staging database!

## Teacher's Viva Tip:

If your examiner asks, *"Why didn't you just import the Excel file directly into the final Data Warehouse?"* You should confidently answer: *"Because legacy Excel files often contain data*

*quality issues like missing values, wrong formats, or duplicates. By loading it into a Staging Area first, we can run SQL scripts to clean and validate the data without risking the integrity of our core Data Warehouse."*

| | |
|---|---|
| 17. | (A)Implement k-means clustering using Python/R Tool (B)Show prediction Using Linear Regression |

****find the ans in above Qs****

| | |
|---|---|
| 20. | **(A)** Create the Data staging area for the selected database.<br>**(B)** Perform the Extraction Transformation and Loading (ETL) process to construct the database in the SQL Server. |

****refer Q16 for staging and Q11 for ETL****